

```
//  
=====
```

NOA DIGIT TRADE - APP.JS FINAL
Supabase Auth + Profiles + Orders + Paiement
+ Litiges
BUY / SELL + Validation réseaux + Paiement
Orange Money
Déclaration paiement via RPC sécurisé
Supabase

```
//  
=====
```

const SUPABASE_URL =
 'https://vowafwsvrjpkhkocptih.supabase.co';

const SUPABASE_KEY =

'sb_publishable_umla4749av8722xus6aQHw_In
f8b-PC';

```
if (!window.supabase) {  
  console.error(  
    "Supabase JS n'a pas été chargé."  
  );  
}
```

```
const supabaseClient =  
  window.supabase.createClient(  
    SUPABASE_URL,  
    SUPABASE_KEY  
  );
```

```
//  
=====
```

=====

```
// CONFIGURATION  
//  
=====
```

=====

```
const CONFIG = {  
  
  buyRate: 600,  
  
  sellRate: 570,  
  
  minOrder: 2000,  
  
  maxOrder: 50000,  
  
  networks: {  
  
    trc20: {  
      name: 'USDT TRC20',  
      fee: 2  
    },  
  
    bp20: {  
      name: 'USDT BEP20',  
      fee: 0  
    }  
  }  
}
```

```
},
```

```
depositAddresses: {
```

```
  trc20:
```

```
    'THmJUvPgBNa7uyDpPKrmer34JtA5w743vA',
```

```
  bp20:
```

```
    '0x9a4248E584B1e27f4d37ce7Ae355Ca004e4  
    39D72'
```

```
},
```

```
payment: {
```

```
  method:
```

```
    'orange_money',
```

```
  number:
```

```
    '74602553',
```

displayNumber:

'74 60 25 53'

},

binance: {

id:

'37467149'

},

balances: {

usdt: 0,

fcfa: 0

}

};

```
//  
=====
```

=====

```
// ÉTAT APPLICATION  
//  
=====
```

=====

```
  
let currentUser = null;  
  
let currentProfile = null;  
  
let currentOrder = null;  
  
let selectedProofFile = null;  
  
let currentExchangeType = 'buy';  
  
let currentNetwork = 'trc20';  
  
let applicationInitializing = false;  
  
let authListenerReady = false;
```

```
//  
=====
```

=====

```
// UTILITAIRES  
//  
=====
```

=====

```
const $ = id =>  
  document.getElementById(id);
```

```
function showMessage(  
  message,  
  type = 'info'  
) {
```

```
  const box =  
    $('appMessage');
```

```
  if (!box) {
```

```
    return;
}

box.textContent =
    String(message || "");

box.className =
    type;

box.id =
    'appMessage';
}

function hideMessage() {

    const box =
        $('appMessage');

    if (!box) {
        return;
    }
}
```



```
box.textContent =  
    "
```

```
box.className =  
    "  
}
```

```
function formatNumber(  
    value,  
    max = 0  
) {
```

```
    const n =  
        Number(value);
```

```
    return (  
        Number.isFinite(n)  
        ? n  
        : 0  
    ).toLocaleString(  
        'fr-FR',  
        {
```

```
        minimumFractionDigits: 0,  
        maximumFractionDigits: max  
    }  
);  
}
```

```
function formatDate(value) {
```

```
    if (!value) {  
        return '-';  
    }
```

```
    const d =  
        new Date(value);
```

```
    if (  
        Number.isNaN(  
            d.getTime()  
        )  
    ) {  
        return '-';  
    }
```

```
return d.toLocaleString(
  'fr-FR',
  {
    day: '2-digit',
    month: '2-digit',
    year: 'numeric',
    hour: '2-digit',
    minute: '2-digit'
  }
);
}
```

```
function normalizePhone(phone) {

  let digits =
    String(phone || "")
      .replace(
        /\D/g,
        ""
      );
}
```

```
if (digits.startsWith('00226')) {  
  
    digits =  
        digits.slice(5);  
  
} else if (digits.startsWith('226')) {  
  
    digits =  
        digits.slice(3);  
}  
  
if (!digits) {  
  
    return " ";  
}  
  
return `+226${digits}`;  
}
```

```
function escapeHtml(value) {  
  
    return String(
```

```
value ?? "  
)  
.replace(  
  /&/g,  
  '&amp;';  
)  
.replace(  
  /</g,  
  '&lt;';  
)  
.replace(  
  />/g,  
  '&gt;';  
)  
.replace(  
  /"/g,  
  '&quot;';  
)  
.replace(  
  /'/g,  
  '&#039;';  
);  
}
```

```
function getSupabaseErrorMessage(  
  error  
) {  
  
  return (  
    error?.message ||  
    error?.error_description ||  
    error?.details ||  
    error?.hint ||  
    'Erreur inconnue.'  
  );  
}
```

```
//
```

```
=====
```

```
=====
```

```
// VALIDATION ADRESSES / RÉSEAUX
```

```
//
```

```
=====
```

```
=====
```

```
function normalizeWalletAddress(  
  value  
) {  
  
  return String(  
    value || "  
  )  
    .trim()  
    .replace(  
      /\s+/g,  
      "  
    );  
}
```

```
function validateWalletForNetwork(  
  wallet,  
  network  
) {  
  
  const address =  
    normalizeWalletAddress(  

```

```
wallet
);

if (!address) {

  return {
    valid: false,
    level: 'error',
    message:
      'Veuillez saisir une adresse de portefeuille
      USDT.'
  };
}

if (
  network ===
  'trc20'
){

  const trc20Regex =
    /^T[1-9A-HJ-NP-Za-km-z]{33}$/;
```



```
if (
  !trc20Regex.test(
    address
  )
){

  return {
    valid: false,
    level: 'error',
    message:
      'Cette adresse ne correspond pas au
format TRC20. Vérifiez que vous avez choisi
TRC20 et copié une adresse TRON
commençant par T.'
  };
}

return {
  valid: true,
  level: 'ok',
```

```
    message:
      'Adresse compatible avec le format
TRC20.'
    };
  }

  if (
    network ===
    'bp20'
  ){

    const bep20Regex =
      /^0x[a-fA-FO-9]{40}$/;

    if (
      !bep20Regex.test(
        address
      )
    ){

      return {
```

```
valid: false,  
level: 'error',  
message:
```

```
    'Cette adresse ne correspond pas au  
format BEP20. Vérifiez que vous avez choisi  
BEP20 et copié une adresse commençant par  
0x.'
```

```
    };  
}
```

```
return {  
    valid: true,  
    level: 'warning',  
    message:
```

```
    'Adresse compatible avec le format  
BEP20. Vérifiez qu'elle est bien destinée au  
réseau BNB Smart Chain.'
```

```
    };  
}
```

```
return {
```

```
    valid: false,  
    level: 'error',  
    message:  
        'Réseau non reconnu.'  
    };  
}
```

```
function validateNoaDepositAddress(  
    network  
) {  
  
    const address =  
        String(  
            CONFIG.depositAddresses?.[network] ||  
            ""  
        ).trim();  
  
    if (!address) {  
  
        return {  
            valid: false,
```

```
    message:
      `L'adresse de dépôt NOA DIGIT TRADE
      n'est pas configurée pour
      ${CONFIG.networks[network]?.name ||
      network}.`
    };
  }
}
```

```
const validation =
  validateWalletForNetwork(
    address,
    network
  );
```

```
if (
  !validation.valid
){
```

```
  return {
    valid: false,
    message:
```

`ERREUR DE CONFIGURATION : l'adresse
de dépôt NOA configurée ne correspond pas
au format du réseau

`${CONFIG.networks[network]?.name ||
network}`. La vente est bloquée pour votre
sécurité.`

```
};  
}
```

```
return {  
  valid: true,  
  message:  
    validation.message  
};  
}
```

```
//
```

```
=====
```

```
=====
```

```
// NAVIGATION
```

```
//
```

```
=====
=====
```

```
function showAuthPage() {

    $('authPage')?.classList.add(
        'active'
    );

    $('appPage')?.classList.remove(
        'active'
    );

    $('bottomNav')?.classList.add(
        'hidden'
    );
}
```

```
function showAppPage() {

    $('authPage')?.classList.remove(
        'active'
```

```
);

$('appPage')?.classList.add(
  'active'
);

$('bottomNav')?.classList.remove(
  'hidden'
);
}
```

```
function showSubPage(
  pageId
){
```

```
  document
    .querySelectorAll(
      '.sub-page'
    )
    .forEach(page => {

      page.classList.add(
```



```
        'hidden'  
    );  
  
    page.classList.remove(  
        'active'  
    );  
  
});
```

```
const target =  
    $(pageld);
```

```
if (target) {  
  
    target.classList.remove(  
        'hidden'  
    );  
  
    target.classList.add(  
        'active'  
    );  
}
```

```
}
```

```
document  
  .querySelectorAll(  
    '.nav-btn'  
  )  
  .forEach(button => {  
  
    button.classList.toggle(  
      'active',  
      button.dataset.page ===  
        pageId  
    );  
  
  });  
  
window.scrollTo({  
  top: 0,  
  behavior: 'smooth'  
});
```

```
if (  
    pageId ===  
    'ordersPage'  
) {  
  
    loadOrderHistory();  
}  
  
if (  
    pageId ===  
    'supportPage'  
) {  
  
    loadDisputes();  
  
    loadOrdersForDispute();  
}  
}  
  
//
```

```
=====

// AUTH FORMULAIRES
//

=====

function showLoginForm() {

    $('loginTab').classList.add(
        'active'
    );

    $('registerTab').classList.remove(
        'active'
    );

    $('loginForm').classList.add(
        'active'
    );

    $('registerForm').classList.remove(
        'active'
    );
}
```

```
);  
}
```

```
function showRegisterForm() {
```

```
    $('loginTab').classList.remove(  
        'active'  
    );
```

```
    $('registerTab').classList.add(  
        'active'  
    );
```

```
    $('loginForm').classList.remove(  
        'active'  
    );
```

```
    $('registerForm').classList.add(  
        'active'  
    );  
}
```

```
function showForgotPasswordForm() {  
  
    $('loginForm')?.classList.remove(  
        'active'  
    );  
  
    $('registerForm')?.classList.remove(  
        'active'  
    );  
  
    $('forgotPasswordForm')?.classList.add(  
        'active'  
    );  
}
```

```
async function sendPasswordReset(  
    event  
) {  
  
    event.preventDefault();
```

```
hideMessage();
```

```
const email =  
  (  
    $('forgotPasswordEmail')  
      ?.value  
      .trim() || "  
  ).toLowerCase();
```

```
if (!email) {
```

```
  return showMessage(  
    'Veuillez saisir votre adresse email.',  
    'error'  
  );  
}
```

```
const button =  
  event.submitter ||  
  $('forgotPasswordForm')
```

```
?.querySelector(  
  'button[type="submit"]'  
);  
  
const original =  
  button?.textContent;  
  
if (button) {  
  
  button.disabled =  
    true;  
  
  button.textContent =  
    'Envoi en cours...';  
}  
  
try {  
  
  const {  
    error  
  } =  
    await supabaseClient.auth
```



```
.resetPasswordForEmail(  
    email,  
    {  
  
        redirectTo:  
            window.location.origin +  
            window.location.pathname  
  
    }  
);  
  
if (error) {  
  
    throw error;  
}  
  
showMessage(  
    'Un email de réinitialisation a été envoyé  
si ce compte existe.',  
    'success'  
);
```

```
showLoginForm();

} catch (error) {

  console.error(
    'Erreur réinitialisation :',
    error
  );

  showMessage(
    getSupabaseErrorMessage(
      error
    ) ||
    "Impossible d'envoyer l'email de
réinitialisation.",
    'error'
  );

} finally {

  if (button) {
```

```
        button.disabled =  
            false;  
  
        button.textContent =  
            original ||  
            'Envoyer le lien de réinitialisation';  
    }  
}  
}
```

```
function getPasswordInput(  
    toggleBtn,  
    targetId  
) {  
  
    if (targetId) {  
  
        const directInput =  
            $(targetId);  
  
        if (directInput) {
```

```
        return directInput;
    }
}

const explicitTarget =
    toggleBtn?.getAttribute?.(
        'data-target'
    ) ||
    toggleBtn?.dataset?.target;

if (explicitTarget) {

    const id =
        explicitTarget.replace(
            /^#/,
            ''
        );

    const explicitInput =
        $(id);
```

```
if (explicitInput) {  
  
    return explicitInput;  
}  
}  
  
const form =  
    toggleBtn?.closest?.(  
        'form'  
    );  
  
const localInput =  
    toggleBtn?.parentElement?.querySelector?.(  
        'input[type="password"]', input[type="text"]  
    [data-password]  
    ) ||  
    toggleBtn?.closest?.(  
        '.password-field, .password-wrapper,  
.input-wrapper, .form-group, .field'  
    )?.querySelector?.(  
        'input[type="password"]', input[type="text"]  
    [data-password])
```

```
);

if (localInput) {

    return localInput;
}

const formPassword =
    form?.querySelector?.(
        'input[type="password"],
input[id="Password"], input[name="password"]'
    );

if (formPassword) {

    return formPassword;
}

return null;
}
```

```
function togglePassword(  
  targetId,  
  toggleBtn  
) {  
  
  const input =  
    getPasswordInput(  
      toggleBtn,  
      targetId  
    );  
  
  if (!input) {  
  
    console.warn(  
      'Champ mot de passe introuvable.'  
    );  
  
    return false;  
  }  
  
  const isHidden =
```

```
input.type ===  
'password';
```

```
input.type =  
isHidden  
  ? 'text'  
  : 'password';
```

```
if (toggleBtn) {
```

```
  toggleBtn.textContent =  
    isHidden  
      ? '🙈'  
      : '👁';
```

```
  toggleBtn.setAttribute(  
    'aria-label',  
    isHidden  
      ? 'Masquer le mot de passe'  
      : 'Afficher le mot de passe'  
  );
```



```
}

    return true;
}

/*
 * Disponible aussi pour les boutons HTML
 * utilisant onclick="togglePassword(...)"
 */
window.togglePassword =
    togglePassword;

function initPasswordToggles() {

    function toggleFromButton(button) {

        if (!button) {
            return;
        }
    }
}
```

```
const targetId =
  button.getAttribute('data-password-
target') ||
  button.getAttribute('data-target') ||
  button.dataset?.passwordTarget ||
  button.dataset?.target;

let input = null;

if (targetId) {
  input = document.getElementById(
    targetId.replace(/^#/,"")
  );
}

if (!input) {
  const wrapper =
    button.closest('.password-wrapper');

  input =
    wrapper?.querySelector(
      'input'
    ) || null;
```

```
}
```

```
if (!input) {  
  return;  
}
```

```
const showPassword =  
  input.type === 'password';
```

```
input.type =  
  showPassword  
    ? 'text'  
    : 'password';
```

```
button.textContent =  
  showPassword  
    ? '🙈'  
    : '👁️';
```

```
button.setAttribute(  
  'aria-label',  
  showPassword  
    ? 'Masquer le mot de passe'
```

```
        : 'Afficher le mot de passe'
    );
}

function bindPasswordButtons() {

    document
        .querySelectorAll(
            'password-toggle, [data-password-
target]'
        )
        .forEach(
            button => {

                if (
                    button.dataset.passwordToggleBound
===
                    'true'
                ){
                    return;
                }
            }
        )
    }
```

```
button.dataset.passwordToggleBound =  
    'true';  
  
button.addEventListener(  
    'click',  
    event => {  
  
        event.preventDefault();  
        event.stopPropagation();  
  
        toggleFromButton(  
            event.currentTarget  
        );  
    },  
    true  
);  
}  
);  
}  
  
/*  
* Liaison immédiate aux boutons déjà
```

présents.

```
*/
```

```
bindPasswordButtons();
```

```
/*
```

* Certains éléments peuvent être recréés dynamiquement.

* On les relie automatiquement dès qu'ils apparaissent.

```
*/
```

```
const observer =
```

```
  new MutationObserver(
```

```
    () => {
```

```
      bindPasswordButtons();
```

```
    }
```

```
  );
```

```
observer.observe(
```

```
  document.documentElement,
```

```
  {
```

```
    childList: true,
```

```
    subtree: true
```

```
}  
);  
}
```

```
function initWhatsAppButton() {
```

```
/*
```

```
 * Le bouton est créé directement par  
JavaScript afin  
 * de rester présent sur toutes les pages de  
l'application.
```

```
*/
```

```
if (
```

```
    document.getElementById(  
        'floatingWhatsAppButton'  
    )
```

```
){
```

```
    return;
```

```
}
```

```
const style =  
  document.createElement(  
    'style'  
  );  
  
style.textContent =  
  `  
    #floatingWhatsAppButton {  
      position: fixed;  
      right: 18px;  
      bottom: 88px;  
      width: 58px;  
      height: 58px;  
      border: none;  
      border-radius: 50%;  
      background: #25D366;  
      color: #ffffff;  
      font-size: 30px;  
      line-height: 58px;  
      text-align: center;  
      box-shadow: 0 6px 18px rgba(0, 0, 0, .28);  
      z-index: 99999;  
      cursor: pointer;  
    }  
  `;
```



```
}

#floatingWhatsAppButton:active {
  transform: scale(.96);
}

`;

document.head.appendChild(
  style
);

const button =
  document.createElement(
    'button'
  );

button.id =
  'floatingWhatsAppButton';

button.type =
  'button';
```

```
button.setAttribute(  
    'aria-label',  
    'Contacter le support sur WhatsApp'  
);
```

```
button.title =  
    'Contacter le support sur WhatsApp';
```

```
button.textContent =  
    '💬';
```

```
button.addEventListener(  
    'click',  
    () => {  
  
        window.open(  
            'https://wa.me/22662591922',  
            '_blank',  
            'noopener,noreferrer'  
        );  
    }  
);
```

```
document.body.appendChild(  
  button  
);  
}
```

```
function copyToClipboard(  
  text,  
  button  
) {
```

```
  if (!text) {
```

```
    return;  
  }
```

```
  const originalLabel =  
    button?.textContent;
```

```
  const onCopied = () => {
```

```
if (!button) {  
  
    return;  
}  
  
button.textContent =  
    '✅ Copié !';  
  
setTimeout(  
    () => {  
  
        button.textContent =  
            originalLabel ||  
            '📋 Copier';  
  
    },  
    2000  
);  
};  
  
if (
```

```
navigator.clipboard &&  
window.isSecureContext  
) {
```

```
    navigator.clipboard  
      .writeText(text)  
      .then(onCopied)  
      .catch(  
        () => {
```

```
          fallbackCopy(  
            text  
          );
```

```
          onCopied();  
        }  
      );
```

```
    return;  
  }
```

```
  fallbackCopy(text);
```

```
onCopied();  
}
```

```
function fallbackCopy(text) {
```

```
  const textarea =  
    document.createElement(  
      'textarea'  
    );
```

```
  textarea.value =  
    text;
```

```
  textarea.style.position =  
    'fixed';
```

```
  textarea.style.opacity =  
    '0';
```

```
  document.body.appendChild(  
    textarea
```

```
);

textarea.select();

try {

    document.execCommand(
        'copy'
    );

} catch (error) {

    console.error(
        'Erreur copie presse-papiers :',
        error
    );
}

document.body.removeChild(
    textarea
);
```

```
}

//
=====

=====
// PROFIL
//
=====

=====

async function loadUserProfile() {

  if (
    !currentUser?.id
  ) {
    return null;
  }

  const metadata =
    currentUser.user_metadata ||
    {};
```



```
const fallback = {  
  
  id:  
    currentUser.id,  
  
  full_name:  
    metadata.full_name ||  
    metadata.name ||  
    currentUser.email  
      ?.split('@')[0] ||  
    'Utilisateur',  
  
  phone:  
    metadata.phone ||  
    '',  
  
  country:  
    metadata.country ||  
    'Burkina Faso',  
  
  role:
```

```
      'user'

    };

    try {

      const {
        data: profile,
        error
      } =
        await supabaseClient
          .from('profiles')
          .select(

            'id,full_name,phone,country,referral_code,inter
            nal_balance'
          )
          .eq(
            'id',
            currentUser.id
          )
          .maybeSingle();
```

```
// Un profil peut déjà exister ici avec
seulement
// l'email (créé automatiquement par le
trigger
// Supabase à l'inscription). On complète
toujours
// les infos (nom, téléphone, pays) via upsert
plutôt
// que de s'arrêter dès qu'une ligne existe.
const needsUpsert =
  !error &&
  (
    !profile ||
    !profile.full_name ||
    !profile.phone
  );

if (
  !error &&
  profile &&
```

```
!needsUpsert
){

  currentProfile =
    profile;

  updateUserInterface();

  return profile;
}
```

```
if (needsUpsert) {

  const {
    data: upserted,
    error: upsertError
  } =
    await supabaseClient
      .from('profiles')
      .upsert(
        {
```

```
id:
  currentUser.id,

full_name:
  fallback.full_name,

phone:
  fallback.phone,

country:
  fallback.country

},
{
  onConflict:
    'id'
}
)
.select(

'id,full_name,phone,country,referral_code,inter
nal_balance'
)
```

```
.maybeSingle());
```

```
if (  
  !upsertError &&  
  upserted  
) {
```

```
  currentProfile =  
    upserted;
```

```
  updateUserInterface();
```

```
  return upserted;  
}
```

```
if (upsertError) {
```

```
  console.warn(  
    'Profil non créé :',  
    upsertError  
  );
```

```
    }

    } else if (error) {

        console.warn(
            'Profil non lisible :',
            error
        );
    }

    } catch (error) {

        console.warn(
            'Profil non disponible :',
            error
        );
    }

    currentProfile =
        fallback;

    updateUserInterface();
```

```
    return currentProfile;  
  }
```

```
function updateUserInterface() {
```

```
  if (!currentUser) {  
    return;  
  }
```

```
  const p =  
    currentProfile || {};
```

```
  const m =  
    currentUser.user_metadata ||  
    {};
```

```
  const name =  
    p.full_name ||
```



```
m.full_name ||  
m.name ||  
'Utilisateur';
```

```
const phone =  
p.phone ||  
m.phone ||  
"";
```

```
const country =  
p.country ||  
m.country ||  
'Burkina Faso';
```

```
if ($('#userName')) {  
  
    $('#userName').textContent =  
        name;  
}
```

```
if ($('#userCountry')) {
```

```
    $('#userCountry').textContent =  
        '🇲🇻' + country;  
}
```

```
if ($('#profileName')) {
```

```
    $('#profileName').value =  
        name;  
}
```

```
if ($('#profilePhone')) {
```

```
    $('#profilePhone').value =  
        phone;  
}
```

```
if ($('#profileCountry')) {
```

```
    $('profileCountry').value =  
        country;  
}
```

```
if ($('#referralCodeDisplay')) {  
  
    $('#referralCodeDisplay').value =  
        p.referral_code ||  
        'Génération en cours...';  
}
```

```
if ($('#referralBalanceDisplay')) {  
  
    const balance =  
        p.internal_balance ||  
        0;  
  
    $('#referralBalanceDisplay').value =  
        new Intl.NumberFormat('fr-  
FR').format(balance) +
```

```
        'FCFA';  
    }  
}  
  
async function initializeApplication() {  
  
    if (  
        applicationInitializing  
    ){  
        return;  
    }  
  
    applicationInitializing =  
        true;  
  
    try {  
  
        await loadUserProfile();  
  
        await loadAppSettings();  
    }  
}
```

```
showAppPage();

updateUserInterface();

updateRatesUI();

updateCalculator();

} catch (error) {

    console.error(
        'Erreur initialisation :',
        error
    );

    showAppPage();

} finally {

    applicationInitializing =
        false;
}
```

```
}

//
=====

=====
// INSCRIPTION
//
=====

=====

async function registerUser(
  event
){

  event.preventDefault();

  hideMessage();

  const name =
    $('registerName')
    ?.value
```

```
.trim() || '';
```

```
const phone =  
  normalizePhone(  
    $('registerPhone')  
      ?.value  
  );
```

```
const country =  
  $('registerCountry')  
    ?.value  
    .trim() ||  
  'Burkina Faso';
```

```
const referralCode =  
  $('registerReferralCode')  
    ?.value  
    .trim()  
    .toUpperCase() || '';
```

```
const email =  
  (  
    $('registerEmail')  
      ?.value  
      .trim() ||  
    "  
  ).toLowerCase();
```

```
const password =  
  $('registerPassword')  
    ?.value || "";
```

```
const confirmPassword =  
  $('registerPasswordConfirm')  
    ?.value || "";
```

```
if (!name) {  
  
  return showMessage(  

```



```
        'Veuillez saisir votre nom et prénom.',  
        'error'  
    );  
}
```

```
if (!phone) {  
  
    return showMessage(  
        'Veuillez saisir votre numéro de  
téléphone.',  
        'error'  
    );  
}
```

```
if (  
    !/^+226\d{8}$/.test(  
        phone  
    )  
) {  
  
    return showMessage(  
        'Veuillez saisir votre numéro de  
téléphone.',  
        'error'  
    );  
}
```

```
        'Veuillez saisir un numéro du Burkina Faso  
valide de 8 chiffres.',
```

```
        'error'  
    );  
}
```

```
if (!email) {
```

```
    return showMessage(  
        'Veuillez saisir votre adresse email.',  
        'error'  
    );  
}
```

```
if (  
    password.length <  
    6  
) {
```

```
    return showMessage(  
        'Le mot de passe doit contenir au moins 6
```

```
caractères.',
    'error'
);
}

if (
    password !==
    confirmPassword
){

    return showMessage(
        'Les deux mots de passe ne correspondent
pas.',
        'error'
    );
}

const button =
    event.submitter ||
    $('registerForm')
    ?.querySelector(
```

```
    'button[type="submit"]'  
  );
```

```
const original =  
  button?.textContent;
```

```
if (button) {
```

```
  button.disabled =  
    true;
```

```
  button.textContent =  
    'Création du compte...';  
}
```

```
try {
```

```
  const {  
    data,  
    error
```

```
} =  
  await supabaseClient.auth  
    .signup({  
  
      email,  
  
      password,  
  
      options: {  
  
        data: {  
  
          full_name:  
            name,  
  
          phone:  
            phone,  
  
          country:  
            country,  
  
          referral_code_used:  
            referralCode
```

```
}
```

```
}
```

```
});
```

```
if (error) {  
  throw error;  
}
```

```
if (!data?.user) {
```

```
  throw new Error(  
    "Le compte n'a pas pu être créé."  
  );  
}
```

```
currentUser =  
  data.user;
```

```
if (data.session) {  
  
    await initializeApplication();  
  
    showMessage(  
        'Compte créé avec succès. Bienvenue sur  
NOA DIGIT TRADE !',  
        'success'  
    );  
  
} else {  
  
    showMessage(  
        'Compte créé avec succès. Vérifiez votre  
email puis connectez-vous.',  
        'success'  
    );  
  
    showLoginForm();  
}
```

```
if ($('#loginEmail')) {  
  
    $('#loginEmail').value =  
        email;  
}  
}  
  
} catch (error) {  
  
    console.error(  
        'Erreur inscription :',  
        error  
    );  
  
    let message =  
        getSupabaseErrorMessage(  
            error  
        );  
  
    if (  
        /already registered|user already
```



```
registered||already exists/i
    .test(message)
){

    message =
        'Cette adresse email est déjà utilisée.';
}

showMessage(
    message,
    'error'
);

} finally {

    if (button) {

        button.disabled =
            false;

        button.textContent =
            original ||
```

```
        'Créer mon compte';
    }
}

//
=====

=====
// CONNEXION
//
=====

=====

async function loginUser(
    event
){

    event.preventDefault();

    hideMessage();
```

```
const email =
(
  $('loginEmail')
    ?.value
    .trim() ||
  ""
).toLowerCase();

const password =
  $('loginPassword')
    ?.value || "";

if (
  !email ||
  !password
){

  return showMessage(
    'Veuillez remplir tous les champs.',
    'error'
  );
```

```
}
```

```
const button =  
  event.submitter ||  
  $('loginForm')  
    ?.querySelector(  
      'button[type="submit"]'  
    );
```

```
const original =  
  button?.textContent;
```

```
if (button) {  
  
  button.disabled =  
    true;  
  
  button.textContent =  
    'Connexion...';  
}
```

```
try {  
  
  const {  
    data,  
    error  
  } =  
    await supabaseClient.auth  
      .signInWithPassword({  
  
        email,  
  
        password  
  
      });  
  
  if (error) {  
    throw error;  
  }  
}
```

```
currentUser =  
  data.user;
```

```
await initializeApplication();
```

```
showMessage(  
  'Connexion réussie.',  
  'success'  
);
```

```
} catch (error) {
```

```
  console.error(  
    'Erreur connexion :',  
    error  
  );
```

```
let message =  
  getSupabaseErrorMessage(  
    error
```

```
);
```

```
if (  
    /invalid login credentials/i  
    .test(message)  
) {
```

```
    message =  
        'Email ou mot de passe incorrect.';  
}
```

```
if (  
    /email not confirmed/i  
    .test(message)  
) {
```

```
    message =  
        "Votre adresse email n'est pas encore  
confirmée.";  
}
```

```
    showMessage(  
        message,  
        'error'  
    );  
  
    } finally {  
  
        if (button) {  
  
            button.disabled =  
                false;  
  
            button.textContent =  
                original ||  
                'Se connecter';  
        }  
    }  
}  
  
//  
=====
```



```
=====
// DÉCONNEXION
//
=====

=====

async function logoutUser() {

  try {

    await supabaseClient.auth
      .signOut();

  } catch (error) {

    console.error(
      'Erreur déconnexion :',
      error
    );
  }

  currentUser =
```

```
        null;

        currentProfile =
            null;

        currentOrder =
            null;

        showAuthPage();

        showLoginForm();

        hideMessage();
    }

    //
    =====
    =====
    // PARAMÈTRES APPLICATION
    //
    =====
```

```
=====
```

```
async function loadAppSettings() {
```

```
  try {
```

```
    const {
```

```
      data,
```

```
      error
```

```
    } =
```

```
      await supabaseClient
```

```
        .from('app_settings')
```

```
        .select("")
```

```
        .eq(
```

```
          'id',
```

```
          1
```

```
        )
```

```
        .maybeSingle();
```

```
    if (error) {
```

```
      console.error(
```

```
        'Erreur app_settings :',  
        error  
    );  
  
    return false;  
}  
  
if (!data) {  
  
    console.warn(  
        'Aucune configuration app_settings  
trouvée.'  
    );  
  
    return false;  
}  
  
const buyRate =  
    Number(  
        data.buy_rate  
    );
```

```
const sellRate =  
  Number(  
    data.sell_rate  
  );
```

```
const minOrder =  
  Number(  
    data.min_order ??  
    data.min_order_cfa  
  );
```

```
const maxOrder =  
  Number(  
    data.max_order ??  
    data.max_order_cfa  
  );
```

```
const trc20Fee =
```

```
Number(  
  data.trc20_fee ??  
  data.trc20_fee_usdt  
);
```

```
const bp20Fee =  
  Number(  
    data.bp20_fee ??  
    data.bp20_fee_usdt  
  );
```

```
const usdtBalance =  
  Number(  
    data.usdt_balance  
  );
```

```
const fcfaBalance =  
  Number(  
    data.fcfa_balance  
  );
```

```
if (  
    Number.isFinite(  
        buyRate  
    ) &&  
    buyRate > 0  
) {  
  
    CONFIG.buyRate =  
        buyRate;  
}
```

```
if (  
    Number.isFinite(  
        sellRate  
    ) &&  
    sellRate > 0  
) {  
  
    CONFIG.sellRate =  
        sellRate;
```

```
}
```

```
if (  
    Number.isFinite(  
        minOrder  
    ) &&  
    minOrder >= 0  
) {
```

```
    CONFIG.minOrder =  
        minOrder;  
}
```

```
if (  
    Number.isFinite(  
        maxOrder  
    ) &&  
    maxOrder >= 0  
) {
```

```
    CONFIG.maxOrder =
```



```
        maxOrder;
    }

    if (
        Number.isFinite(
            trc20Fee
        ) &&
        trc20Fee >= 0
    ){

        CONFIG.networks
            .trc20
            .fee =
                trc20Fee;
    }

    if (
        Number.isFinite(
            bp20Fee
        ) &&
        bp20Fee >= 0
    )
```

```
) {
```

```
    CONFIG.networks
```

```
        .bp20
```

```
        .fee =
```

```
        bp20Fee;
```

```
}
```

```
if (
```

```
    Number.isFinite(
```

```
        usdtBalance
```

```
    ) &&
```

```
    usdtBalance >= 0
```

```
) {
```

```
    CONFIG.balances.usdt =
```

```
        usdtBalance;
```

```
}
```

```
if (
```

```
    Number.isFinite(
```

```
        fcfaBalance
    ) &&
    fcfaBalance >= 0
) {

    CONFIG.balances.fcfa =
        fcfaBalance;
}

return true;

} catch (error) {

    console.error(
        'Erreur lecture paramètres :',
        error
    );

    return false;
}
}
```

```
//  
=====
```

=====

```
// TARIFS  
//  
=====
```

=====

```
function updateRatesUI() {  
  
  if ($('#homeBuyRate')) {  
  
    $('#homeBuyRate').textContent =  
      formatNumber(  
        CONFIG.buyRate  
      );  
  }  
  
  if ($('#homeSellRate')) {  
  
    $('#homeSellRate').textContent =
```

```
formatNumber(  
    CONFIG.sellRate  
);  
}
```

```
if ($('#homeMinOrder')) {
```

```
    $('#homeMinOrder').textContent =  
        formatNumber(  
            CONFIG.minOrder  
        );  
}
```

```
if ($('#homeMaxOrder')) {
```

```
    $('#homeMaxOrder').textContent =  
        formatNumber(  
            CONFIG.maxOrder  
        );  
}
```

```
if ($('#homeTrc20Fee')) {
```

```
    $('#homeTrc20Fee').textContent =  
        CONFIG.networks.trc20.fee;  
}
```

```
if ($('#homeBp20Fee')) {
```

```
    $('#homeBp20Fee').textContent =  
        CONFIG.networks.bp20.fee;  
}
```

```
if ($('#trc20Fee')) {
```

```
    $('#trc20Fee').textContent =  
        CONFIG.networks.trc20.fee;  
}
```

```
if ($('#bp20Fee')) {
```

```
    $('bp20Fee').textContent =  
        CONFIG.networks.bp20.fee;  
    }  
}  
  
//  
=====
```

=====

```
// CHAMP NUMÉRO ORANGE MONEY POUR  
VENTE  
//  
=====
```

=====

```
function ensureSellPayoutField() {  
  
    const walletInput =  
        $('walletAddress');  
  
    if (!walletInput) {
```

```
    return null;  
  }
```

```
  let field =  
    $('sellPayoutField');
```

```
  if (!field) {
```

```
    field =  
      document.createElement(  
        'div'  
      );
```

```
    field.id =  
      'sellPayoutField';
```

```
    field.className =  
      'field';
```



```
field.innerHTML = `
```

```
<label for="sellPayoutPhone">
```

```
    Numéro Orange Money pour recevoir vos  
    FCFA
```

```
</label>
```

```
<input
```

```
    type="tel"
```

```
    id="sellPayoutPhone"
```

```
    inputmode="numeric"
```

```
    autocomplete="tel"
```

```
    maxlength="8"
```

```
    placeholder="Exemple : 70 00 00 00"
```

```
>
```

```
<div class="small wallet-help">
```



```
    Ce numéro sera utilisé pour vous  
    envoyer le montant en FCFA après réception  
    et vérification de vos USDT.
```

```
</div>
```

```
`;  
`;
```

```
const walletField =  
  walletInput.closest('.field') ||  
  walletInput.parentElement;
```

```
if (  
  walletField?.parentNode  
) {
```

```
  walletField.parentNode.insertBefore(  
    field,  
    walletField.nextSibling  
  );
```

```
  }  
}
```

```
return field;  
}
```

```
//  
=====
```

=====

```
// ADRESSE DÉPÔT NOA  
//  
=====
```

=====

```
function getDepositAddress() {  
  
    return String(  
        CONFIG.depositAddresses?.[  
            currentNetwork  
        ] || "  
    ).trim();  
}  
  
//  
=====
```

=====

```
// INTERFACE VENTE
```

```
//  
=====
```



```
function updateSellDepositUI() {  
  
    if (  
        currentExchangeType !==  
        'sell'  
    ) {  
        return;  
    }  
  
    const address =  
        getDepositAddress();  
  
    const walletInput =  
        $('walletAddress');  
  
    if ($('walletLabel')) {
```

```
$(`walletLabel`).textContent =  
  'Adresse de dépôt NOA DIGIT TRADE';  
}  
  
if (walletInput) {  
  
  walletInput.readOnly =  
    true;  
  
  walletInput.required =  
    false;  
  
  walletInput.value =  
    address;  
  
  walletInput.placeholder =  
    address  
    ? 'Adresse de dépôt NOA DIGIT TRADE'  
    : 'Adresse de dépôt non configurée';  
}
```

```

const help =
  walletInput
    ?.closest('.field')
    ?.querySelector(
      '.wallet-help'
    );

if (help) {

  help.innerHTML =
    address
      ? `⚠ Envoyez vos USDT
<strong>uniquement</strong> à cette adresse
et utilisez exactement le réseau
<strong>${escapeHtml(CONFIG.networks[currentNetwork]?.name || currentNetwork)}</strong>.`
      : `⚠ L'adresse de dépôt n'est pas encore
configurée par l'administrateur.`;
}

```

```
const payoutField =  
  ensureSellPayoutField();
```

```
if (payoutField) {
```

```
  payoutField.style.display =  
    "";
```

```
  }  
}
```

```
//
```

```
=====
```

```
=====
```

```
// MODE ACHAT
```

```
//
```

```
=====
```

```
=====
```

```
function setBuyMode() {
```

```
  currentExchangeType =
```

```
'buy';
```

```
$('#buyTab')?.classList.add(  
  'active'  
);
```

```
$('#sellTab')?.classList.remove(  
  'active'  
);
```

```
if ($('#amountLabel')) {
```

```
  $('#amountLabel').textContent =  
    'Montant à payer';  
}
```

```
if ($('#amountUnit')) {
```

```
  $('#amountUnit').textContent =  
    'FCFA';
```



```
}
```

```
const input =  
  $('amountInput');
```

```
if (input) {
```

```
  input.placeholder =  
    'Ex. 10 000';
```

```
  input.min =  
    String(  
      CONFIG.minOrder  
    );
```

```
  input.max =  
    String(  
      CONFIG.maxOrder  
    );
```

```
  input.step =
```

```
    '1';  
  }  
  
  const walletInput =  
    $('walletAddress');  
  
  if ($('walletLabel')) {  
  
    $('walletLabel').textContent =  
      'Adresse de portefeuille USDT';  
  }  
  
  if (walletInput) {  
  
    walletInput.value =  
      '';  
  
    walletInput.readOnly =  
      false;
```

```
walletInput.required =  
  true;  
  
walletInput.placeholder =  
  'Collez votre adresse USDT ici';  
}  
  
const walletHelp =  
  walletInput  
    ?.closest('.field')  
    ?.querySelector(  
      '.wallet-help'  
    );  
  
if (walletHelp) {  
  
  walletHelp.textContent =  
    " ⚠ Pour un achat, indiquez l'adresse USDT  
sur laquelle vous souhaitez recevoir vos USDT."  
}
```

```
if ($('#paymentMethodField')) {
```

```
    $('#paymentMethodField').style.display =  
        '';  
}
```

```
const payoutField =  
    ensureSellPayoutField();
```

```
if (payoutField) {
```

```
    payoutField.style.display =  
        'none';  
}
```

```
if ($('#exchangeInfo')) {
```

```
    $('#exchangeInfo').innerHTML = `
```

💡 Minimum :

`${formatNumber(CONFIG.minOrder)}` FCFA

Maximum :

`${formatNumber(CONFIG.maxOrder)}` FCFA

Le montant exact et les frais sont affichés
avant la confirmation.

```
`;  
}
```

```
updateCalculator();  
}
```

```
//  
=====
```

=====

```
// MODE VENTE  
//  
=====
```

=====

```
function setSellMode() {  
  
    currentExchangeType =  
        'sell';  
  
    $('buyTab').classList.remove(  
        'active'  
    );  
  
    $('sellTab').classList.add(  
        'active'  
    );  
}
```

```
if ($('#amountLabel')) {  
  
    $('#amountLabel').textContent =  
        'Quantité à vendre';  
}
```

```
if ($('#amountUnit')) {  
  
    $('#amountUnit').textContent =  
        'USDT';  
}
```

```
const input =  
    $('#amountInput');
```

```
if (input) {  
  
    input.placeholder =
```

'Exemple : 10 USDT';

input.min =

'0';

input.max =

'';

input.step =

'any';

}

if (\$('#paymentMethodField')) {

\$('#paymentMethodField').style.display =

'none';

}

if (\$('#exchangeInfo')) {

\$('#exchangeInfo').innerHTML = `



Saisissez la

quantité d'USDT

que vous souhaitez vendre.

Nous vous indiquons l'adresse de dépôt
NOA selon le réseau choisi.

Après réception et vérification des USDT,
nous vous envoyons les FCFA sur votre
Orange Money.

;
}

ensureSellPayoutField();

```
updateSellDepositUI();

updateCalculator();
}

//
=====

=====
// RÉSEAU
//
=====

=====

function selectNetwork(
    network
){

    if (
        !CONFIG.networks[network]
    ){
        return;
```

```
}
```

```
currentNetwork =  
    network;
```

```
$('#trc20Option')  
    ?.classList.toggle(  
        'active',  
        network === 'trc20'  
    );
```

```
$('#bp20Option')  
    ?.classList.toggle(  
        'active',  
        network === 'bp20'  
    );
```

```
if (  
    currentExchangeType ===
```

```
    'sell'  
  ){  
  
    updateSellDepositUI();  
  }
```

```
    updateCalculator();  
  }
```

```
//
```

```
=====
```

```
=====
```

```
// CALCUL
```

```
//
```

```
=====
```

```
=====
```

```
function calculateOrder() {
```

```
    const amount =
```

```
        Number(
```

```
    $('amountInput')  
      ?.value  
    ) || 0;
```

```
const fee =  
  Number(  
    CONFIG  
      .networks[  
        currentNetwork  
      ]?.fee || 0  
  );
```

```
if (  
  currentExchangeType ===  
    'buy'  
) {
```

```
  const amountCfa =  
    amount;
```

```
const grossUsdt =  
  amountCfa /  
  CONFIG.buyRate;
```

```
const netUsdt =  
  Math.max(  
    grossUsdt - fee,  
    0  
  );
```

```
return {  
  
  side:  
    'buy',  
  
  amountCfa:  
    amountCfa,  
  
  usdtAmount:  
    grossUsdt,
```

```
    feeUsdt:
      fee,

    netUsdt:
      netUsdt,

    receiveCfa:
      0,

    rate:
      CONFIG.buyRate

  };
}

const usdtAmount =
  amount;

/*
 * VENTE :
 * Le vendeur envoie toute la quantité d'USDT
```

saisie.

* Les frais sont déduits du montant FCFA
qu'il reçoit.

*/

```
const grossCfa =  
  usdtAmount *  
  CONFIG.sellRate;
```

```
const feeCfa =  
  fee *  
  CONFIG.sellRate;
```

```
const receiveCfa =  
  Math.max(  
    grossCfa - feeCfa,  
    0  
  );
```

```
return {
```



```
side:
  'sell',

amountCfa:
  grossCfa,

usdtAmount:
  usdtAmount,

feeUsdt:
  fee,

feeCfa:
  feeCfa,

netUsdt:
  usdtAmount,

receiveCfa:
  receiveCfa,

rate:
  CONFIG.sellRate
```

```

    };
}

//
=====

=====
// AFFICHAGE CALCULATEUR
//
=====

=====

function updateCalculator() {

    const c =
        calculateOrder();

    if ($('#summaryRate')) {

        $('#summaryRate').textContent =
            `${formatNumber(c.rate)} FCFA / USDT`;
    }
}

```

```
}
```

```
if (  
  currentExchangeType ===  
  'buy'  
) {
```

```
  if ($('#summaryCfa')) {
```

```
    $('#summaryCfa').textContent =  
    `${formatNumber(c.amountCfa)} FCFA`;  
  }
```

```
  if ($('#summaryUsdt')) {
```

```
    $('#summaryUsdt').textContent =  
    `${c.usdtAmount.toFixed(6)} USDT`;  
  }
```

```
  if ($('#summaryFee')) {
```

```
    $('summaryFee').textContent =  
        `${c.feeUsdt} USDT`;  
}
```

```
if ($('#summaryResultLabel')) {
```

```
    $('#summaryResultLabel').textContent =  
        'Vous recevez';  
}
```

```
if ($('#summaryResult')) {
```

```
    $('#summaryResult').textContent =  
        `${c.netUsdt.toFixed(6)} USDT`;  
}
```

```
updateBalanceUI(c);
```

```
    return;
  }

  if ($('#summaryCfa')) {

    $('#summaryCfa').textContent =
      `${formatNumber(c.receiveCfa)} FCFA`;
  }

  if ($('#summaryUsdt')) {

    $('#summaryUsdt').textContent =
      `${c.usdtAmount.toFixed(6)} USDT`;
  }

  if ($('#summaryFee')) {

    const sellFeeCfa =
      Number(c.feeCfa) ||
      (
```

```
        Number(c.feeUsdt) *  
        Number(c.rate)  
    );  
  
    $('summaryFee').textContent =  
        `${formatNumber(sellFeeCfa)} FCFA`;  
}  
  
if ($('#summaryResultLabel')) {  
  
    $('#summaryResultLabel').textContent =  
        'Vous recevez';  
}  
  
if ($('#summaryResult')) {  
  
    $('#summaryResult').textContent =  
        `${formatNumber(c.receiveCfa)} FCFA`;  
}
```

```
updateBalanceUl(c);
}

//
=====

=====
// SOLDE DISPONIBLE
//
=====
=====

function updateBalanceUl(c) {

  const box =
    $('availableBalanceBox');

  const valueEl =
    $('availableBalanceValue');

  const alertEl =
    $('insufficientBalanceAlert');
```

```
const reviewBtn =  
  $('reviewOrderBtn');  
  
if (!box || !valueEl) {  
  
  return;  
}  
  
let available = 0;  
  
let needed = 0;  
  
let unit = " ";  
  
if (  
  currentExchangeType ===  
  'buy'  
) {  
  
  available =
```



```
    Number(  
        CONFIG.balances.usdt  
    ) || 0;  
  
    needed =  
        c.usdtAmount;  
  
    unit =  
        'USDT';  
  
    valueEl.textContent =  
        `${available.toFixed(2)} USDT`;  
  
} else {  
  
    available =  
        Number(  
            CONFIG.balances.fcfa  
        ) || 0;  
  
    needed =  
        c.receiveCfa;
```

```
unit =  
    'FCFA';  
  
valueEl.textContent =  
    `${formatNumber(available)} FCFA`;  
}
```

```
const insufficient =  
    needed > 0 &&  
    needed > available;
```

```
box.classList.toggle(  
    'insufficient',  
    insufficient  
);
```

```
if (alertEl) {  
  
    alertEl.classList.toggle(  
        'visible',
```

```
        insufficient  
    );  
}
```

```
if (reviewBtn) {  
  
    reviewBtn.disabled =  
        insufficient;  
}  
}
```

```
//
```

```
=====
```

```
=====
```

```
// RÉCUPÉRER WALLET CLIENT
```

```
//
```

```
=====
```

```
=====
```

```
function getWalletAddress() {
```

```
const ids = [  
  
  'walletAddress',  
  
  'wallet',  
  
  'walletInput',  
  
  'userWallet',  
  
  'wallet_address'  
  
];
```

```
for (  
  const id of ids  
) {
```

```
  const el =  
    $(id);
```

```
    if (
      el?.value?.trim()
    ){

      return normalizeWalletAddress(
        el.value
      );
    }
  }

  return "";
}

//
=====

=====
// VÉRIFICATION COMMANDE
//
=====

=====
```

```
function reviewOrder() {  
  
    hideMessage();  
  
    if (!currentUser) {  
  
        showMessage(  
            'Vous devez être connecté pour passer une  
commande.',  
            'error'  
        );  
  
        showAuthPage();  
  
        showLoginForm();  
  
        return;  
    }  
  
    const c =  
        calculateOrder();
```

```
if (
    !c.usdtAmount ||
    c.usdtAmount <= 0
){

    return showMessage(

        currentExchangeType ===
        'buy'

        ? 'Veuillez saisir un montant en FCFA.'

        : "Veuillez saisir la quantité d'USDT à
vendre.",

        'error'
    );
}

if (
```

```
currentExchangeType ===  
'buy'  
) {  
  
    if (  
        c.amountCfa <  
        CONFIG.minOrder  
    ) {  
  
        return showMessage(  
            'Le montant minimum est de  
${formatNumber(CONFIG.minOrder)} FCFA.',  
            'error'  
        );  
    }  
  
    if (  
        c.amountCfa >  
        CONFIG.maxOrder  
    ) {  
  
        return showMessage(  

```



```
        'Le montant maximum est de  
        ${formatNumber(CONFIG.maxOrder)} FCFA.',  
        'error'  
    );  
}
```

```
if (  
    c.netUsdt <= 0  
) {
```

```
    return showMessage(  
        'Le montant USDT après frais est  
insuffisant.',  
        'error'  
    );  
}  
}
```

```
let wallet = '';
```

```
let payoutPhone = '';
```

```
//  
=====
```

=====

```
// VENTE  
//  
=====
```

=====

```
  
if (  
    currentExchangeType ===  
    'sell'  
) {  
  
    const minimumUsdt =  
        CONFIG.minOrder /  
        CONFIG.sellRate;  
  
    const maximumUsdt =  
        CONFIG.maxOrder /  
        CONFIG.sellRate;
```

```
if (  
    c.usdtAmount <  
    minimumUsdt  
) {  
  
    return showMessage(  
        `La quantité minimum est de  
        ${minimumUsdt.toFixed(2)} USDT`,  
        'error'  
    );  
}
```

```
if (  
    c.usdtAmount >  
    maximumUsdt  
) {  
  
    return showMessage(  
        `La quantité maximum est de  
        ${maximumUsdt.toFixed(2)} USDT`,
```

```

        'error'
    );
}

if (
    c.receiveCfa <= 0
){

    return showMessage(
        "Le montant à recevoir après frais est
insuffisant.",
        'error'
    );
}

wallet =
    getDepositAddress();

const noaValidation =
    validateNoaDepositAddress(

```

```
        currentNetwork
    );

    if (
        !noaValidation.valid
    ){

        return showMessage(
            noaValidation.message,
            'error'
        );
    }
```

```
payoutPhone =
    normalizePhone(
        $('sellPayoutPhone')
        ?.value
    );
```

```
if (!payoutPhone) {
```

```
        return showMessage(  
            'Veuillez saisir le numéro Orange Money  
sur lequel vous souhaitez recevoir vos FCFA.',  
            'error'  
        );  
    }
```

```
    if (  
        !/^+226\d{8}$/.test(  
            payoutPhone  
        )  
    ){
```

```
        return showMessage(  
            'Veuillez saisir un numéro Orange Money  
valide de 8 chiffres.',  
            'error'  
        );  
    }  
}
```

```
//
=====

=====

// ACHAT
//
=====

=====

else {

    wallet =
        getWalletAddress();

    if (!wallet) {

        return showMessage(
            'Veuillez saisir votre adresse de
portefeuille USDT.',
            'error'
        );
    }
}
```

```
const walletValidation =  
  validateWalletForNetwork(  
    wallet,  
    currentNetwork  
  );
```

```
if (  
  !walletValidation.valid  
) {  
  
  return showMessage(  
    walletValidation.message,  
    'error'  
  );  
}
```

```
if (  
  currentNetwork ===  
  'bp20'
```



```
) {  
  
    showMessage(  
        '⚠ Adresse compatible BEP20. Vérifiez  
une dernière fois que cette adresse est bien  
destinée au réseau BNB Smart Chain avant de  
continuer.',  
        'info'  
    );  
}  
}
```

```
currentOrder = {  
  
    side:  
        currentExchangeType,  
  
    network:  
        currentNetwork,  
  
    amountCfa:  
        c.amountCfa,
```

usdtAmount:
c.usdtAmount,

feeUsdt:
c.feeUsdt,

feeCfa:
c.feeCfa || 0,

netUsdt:
c.netUsdt,

receiveCfa:
c.receiveCfa,

rate:
c.rate,

walletAddress:
wallet,

payoutPhone:

```
payoutPhone,
```

```
paymentMethod:  
  'orange_money'
```

```
};
```

```
renderConfirmation();
```

```
showSubPage(  
  'confirmationPage'  
);  
}
```

```
//
```

```
=====
```

```
=====
```

```
// CONFIRMATION
```

```
//
```

```
=====
```

```
=====
```

```
function renderConfirmation() {
```

```
  const box =
```

```
    $('confirmationSummary');
```

```
  if (
```

```
    !box ||
```

```
    !currentOrder
```

```
  ) {
```

```
    return;
```

```
  }
```

```
  const networkName =
```

```
    CONFIG.networks[
```

```
      currentOrder.network
```

```
    ]?.name ||
```

```
    currentOrder.network ||
```

```
    '-';
```

```
if (
  currentOrder.side ===
  'buy'
){

  box.innerHTML = `

    <div class="summary-row">
      <span>Type</span>
      <strong>Achat USDT</strong>
    </div>

    <div class="summary-row">
      <span>Montant à payer</span>
      <strong>
        ${formatNumber(currentOrder.amountCfa)}
        FCFA
      </strong>
    </div>

    <div class="summary-row">
```

Taux

 \${formatNumber(currentOrder.rate)}

FCFA / USDT

</div>

<div class="summary-row">

 USDT acheté

 \${Number(currentOrder.usdtAmount).toFixed(6)}

 USDT

</div>

<div class="summary-row">

 Frais ré seau

 \${Number(currentOrder.feeUsdt)} USDT

</div>

```
<div class="summary-row">  
  <span>USDT net reçu</span>  
  <strong>
```

```
    ${Number(currentOrder.netUsdt).toFixed(6)}  
    USDT
```

```
  </strong>  
</div>
```

```
<div class="summary-row">  
  <span>Réseau</span>  
  <strong>  
    ${escapeHtml(networkName)}  
  </strong>  
</div>
```

```
<div class="summary-row">  
  <span>Portefeuille de réception</span>  
  <strong class="break-word">
```

```
    ${escapeHtml(currentOrder.walletAddress)}  
  </strong>  
</div>
```

```
<div class="summary-row">
```

```
  <span>Paiement</span>
```

```
  <strong>
```

```
    Orange Money
```

```
  </strong>
```

```
</div>
```

```
<div class="summary-row summary-total">
```

```
  <span>Vous recevez</span>
```

```
  <strong>
```

```
    ${Number(currentOrder.netUsdt).toFixed(6)}
```

```
    USDT
```

```
  </strong>
```

```
</div>
```

```
<div class="warning-box">
```



Vérifiez attentivement le réseau

```
  <strong>${escapeHtml(networkName)}
```

```
</strong>
```

```
  et votre adresse avant de confirmer.
```

```
</div>
```



```
`;  
`
```

```
return;  
}
```

```
box.innerHTML = `
```

```
<div class="summary-row">  
  <span>Type</span>  
  <strong>Vente USDT</strong>  
</div>
```

```
<div class="summary-row">  
  <span>Quantité vendue</span>  
  <strong>
```

```
  ${Number(currentOrder.usdtAmount).toFixed(6)}  
  USDT  
</strong>  
</div>
```

```
<div class="summary-row">
  <span>Taux de vente</span>
  <strong>
    ${formatNumber(currentOrder.rate)} FCFA
  / USDT
  </strong>
</div>
```

```
<div class="summary-row">
  <span>Frais ré seau dé duits</span>
  <strong>
    ${formatNumber(
      Number(currentOrder.feeCfa) ||
      (
        Number(currentOrder.feeUsdt) *
        Number(currentOrder.rate)
      )
    )} FCFA
  </strong>
</div>
```

```
<div class="summary-row">
  <span>Montant brut</span>
```


`\${formatNumber(currentOrder.amountCfa)}

FCFA

</div>

<div class="summary-row">

Réseau

`\${escapeHtml(networkName)}`

</div>

<div class="summary-row">

Adresse de dépôt NOA

<strong class="break-word">

`\${escapeHtml(currentOrder.walletAddress)}`

</div>

<div class="summary-row">

```
<span>Orange Money</span>
<strong>
  ${escapeHtml(currentOrder.payoutPhone
|| '-'')}
</strong>
</div>
```

```
<div class="summary-row summary-total">
  <span>Vous recevrez</span>
  <strong>
    ${formatNumber(currentOrder.receiveCfa)}
    FCFA
  </strong>
</div>
```

```
<div class="warning-box">
  ⚠ <strong>Vérifiez le réseau avant
d'envoyer.</strong>
  <br><br>
  Vous devez envoyer vos USDT uniquement
sur
  <strong>${escapeHtml(networkName)}
```


à l'adresse NOA indiquée ci-dessus.

Une erreur de réseau peut entraîner une
perte des fonds.

</div>

;

}

//

=====

=====

// ANNULER CONFIRMATION

//

=====

=====

function cancelReview() {

currentOrder =

null;

```
showSubPage(  
  'exchangePage'  
);  
}
```

```
//
```

```
=====
```

```
=====
```

```
// CRÉER COMMANDE
```

```
//
```

```
=====
```

```
=====
```

```
async function placeOrder() {
```

```
  hideMessage();
```

```
  if (!currentUser) {
```

```
    showMessage(  
      'Veuillez vous connecter pour continuer.'  
    );  
  }  
}
```

```
    "Votre session a expiré. Veuillez vous  
reconnecter.",
```

```
    'error'
```

```
);
```

```
showAuthPage();
```

```
showLoginForm();
```

```
return;
```

```
}
```

```
if (!currentOrder) {
```

```
    return showMessage(
```

```
        'Aucune commande à enregistrer.',
```

```
        'error'
```

```
    );
```

```
}
```

```
if (
```

```
currentOrder.side ===  
'buy'  
) {  
  
  const validation =  
    validateWalletForNetwork(  
      currentOrder.walletAddress,  
      currentOrder.network  
    );  
  
  if (  
    !validation.valid  
  ) {  
  
    return showMessage(  
      'Commande bloquée : ' +  
      validation.message,  
      'error'  
    );  
  }  
  
  } else {
```



```
const validation =  
  validateNoaDepositAddress(  
    currentOrder.network  
  );  
  
if (  
  !validation.valid  
) {  
  
  return showMessage(  
    validation.message,  
    'error'  
  );  
}  
  
const button =  
  $('placeOrderBtn');
```

```
const originalText =  
  button?.textContent;  
  
if (button) {  
  
  button.disabled =  
    true;  
  
  button.textContent =  
    'Enregistrement...';  
}  
  
try {  
  
  const {  
    data: sessionData,  
    error: sessionError  
  } =  
    await supabaseClient.auth  
      .getSession();
```

```
if (sessionError) {  
    throw sessionError;  
}
```

```
if (  
    !sessionData?.session?.user  
) {
```

```
    throw new Error(  
        "Votre session n'est plus active."  
    );  
}
```

```
currentUser =  
    sessionData.session.user;
```

```
let customerNote =  
    null;
```

```
if (
  currentOrder.side ===
  'buy'
){

  customerNote =
    currentOrder.walletAddress
    ? `Adresse de réception USDT :
    ${currentOrder.walletAddress}`
    : null;

} else {

  customerNote =
    `Adresse de dépôt NOA :
    ${currentOrder.walletAddress} | Numéro
    Orange Money : ${currentOrder.payoutPhone}`;
}

const receiveCfa =
  currentOrder.side ===
```

'sell'

```
? Number(  
  currentOrder.receiveCfa ||  
  0  
)
```

: 0;

const payload = {

```
  user_id:  
    currentUser.id,
```

```
  side:  
    currentOrder.side,
```

```
  network:  
    currentOrder.network,
```

```
  payment_method:  
    currentOrder.paymentMethod ||
```

```
'orange_money',
```

```
amount_cfa:
```

```
Number(  
    currentOrder.amountCfa  
)
```

```
usdt_amount:
```

```
Number(  
    currentOrder.usdtAmount  
)
```

```
fee_usdt:
```

```
Number(  
    currentOrder.feeUsdt  
)
```

```
net_usdt:
```

```
Number(  
    currentOrder.netUsdt  
)
```

```
receive_cfa:
```

```
receiveCfa,
```

```
status:
```

```
  'pending',
```

```
customer_note:
```

```
  customerNote,
```

```
wallet_address:
```

```
  currentOrder.side ===
```

```
  'buy'
```

```
    ? currentOrder.walletAddress
```

```
  : null,
```

```
payout_phone:
```

```
  currentOrder.side ===
```

```
  'sell'
```

```
    ? currentOrder.payoutPhone
```

```
  : null
```

```
};
```

```
const {  
  data,  
  error  
} =  
  await supabaseClient  
    .from('orders')  
    .insert(  
      payload  
    )  
    .select("")  
    .single();
```

```
if (error) {  
  throw error;  
}
```

```
if (!data) {
```



```
    throw new Error(  
      "Supabase n'a retourné aucune  
commande."  
    );  
  }
```

```
    currentOrder.id =  
      data.id;
```

```
    currentOrder.createdAt =  
      data.created_at;
```

```
    currentOrder.status =  
      data.status ||  
      'pending';
```

```
    if (  
      currentOrder.side ===
```

```
        'buy'
    ){

        renderPaymentPage();

        showSubPage(
            'paymentPage'
        );

        showMessage(
            'Commande enregistrée. Effectuez
maintenant le paiement Orange Money.',
            'success'
        );

    } else {

        renderSellPaymentPage();

        showSubPage(
            'paymentPage'
        );
    }
}
```

```
    showMessage(  
        "Commande de vente enregistrée.  
Envoyez maintenant les USDT à l'adresse  
indiquée.",  
        'success'  
    );  
}
```

```
await loadOrderHistory();
```

```
} catch (error) {
```

```
    console.error(  
        'ERREUR CRÉATION COMMANDE :',  
        error  
    );
```

```
showMessage(  
    'Impossible d'enregistrer la commande : ' +
```

```
        getSupabaseErrorMessage(error),
        'error'
    );

    } finally {

        if (button) {

            button.disabled =
                false;

            button.textContent =
                originalText ||
                'Placer la commande';
        }
    }

    //
    =====

    =====

    // PAGE PAIEMENT ACHAT
```

```
// NOUVELLE VERSION COMPATIBLE AVEC
INDEX.HTML

//
=====

function renderPaymentPage() {

  if (
    !currentOrder ||
    currentOrder.side !== 'buy'
  ) {
    return;
  }

  const amount =
    Math.round(
      Number(
        currentOrder.amountCfa
      ) || 0
    );
```

```
if (  
    !amount ||  
    amount <= 0  
) {  
  
    showMessage(  
        'Montant de paiement invalide.',  
        'error'  
    );  
  
    return;  
}
```

```
// -----  
-----  
// DESCRIPTION  
// -----  
-----
```

```
if ($('#paymentDescription')) {
```

```
    $('paymentDescription').textContent =  
        'Votre commande a été enregistrée.  
Effectuez maintenant le paiement Orange  
Money.';  
}
```

```
if ($('qrInstructionText')) {
```

```
    $('qrInstructionText').style.display =  
        '';  
}
```

```
// -----  
-----  
// MONTANT  
// -----  
-----
```

```
if ($('paymentAmountValue')) {
```

```
    $('paymentAmountValue').textContent =
```

```
`${formatNumber(amount)} FCFA`  
}  
  
// -----  
-----  
  
// NUMÉRO ORANGE MONEY  
// IMPORTANT :  
// Le numéro n'est PAS affiché en clair dans  
l'interface.  
// -----  
-----  
  
const paymentNumber =  
  String(  
    CONFIG.payment.number ||  
    "  
  ).replace(  
    /\D/g,  
    "  
  );
```



```
if (!paymentNumber) {  
  
    showMessage(  
        'Le moyen de paiement n'est pas  
correctement configuré.',  
        'error'  
    );  
  
    return;  
}
```

```
// -----  
-----
```

```
// CODE USSD
```

```
// -----  
-----
```

```
const ussdCode =
```

```
    '*144*10*${paymentNumber}*${amount}#';
```

```
// -----
```

```
-----  
// QR CODE  
// -----  
-----  
  
const qrContainer =  
  $('orangeMoneyQr');  
  
if (qrContainer) {  
  
  qrContainer.innerHTML =  
    ”;  
  
  if (  
    typeof QRCode ===  
    'undefined'  
  ) {  
  
    qrContainer.innerHTML = `  
  
    <div class="small center">
```

QR code indisponible.

Utilisez le bouton de paiement ci-dessous.

</div>

`;

} else {

try {

new QRCode(

qrContainer,

{

text:

`tel:\${ussdCode}`,

width:

220,

height:

220,

correctLevel:

QRCodeCorrectLevel.M

}

);

} catch (error) {

console.error(

'Erreur génération QR :',

error

);

qrContainer.innerHTML = `

<div class="small center">

QR code indisponible.

Utilisez le bouton de paiement ci-
dessous.

```
</div>
```

```
    ;  
  }  
}  
}
```

```
// -----
```

```
-----
```

```
// BOUTON PAIEMENT
```

```
// -----
```

```
-----
```

```
const payButton =  
  $('orangeMoneyPayBtn');
```

```
if (payButton) {
```

```
  payButton.style.display =  
    '';
```

```
payButton.textContent =  
    'CLIQUEZ ICI POUR EFFECTUER LE  
    PAIEMENT';  
  
// Éviter plusieurs listeners  
payButton.onclick =  
    null;  
  
payButton.onclick =  
    () => {  
  
        try {  
  
            /*  
             * Encodage du # pour éviter qu'il soit  
interprété  
             * comme un fragment de l'URL.  
             */  
            const encodedCode =  
                ussdCode.replace(
```

```
'#',  
    '%23'  
);
```

```
window.location.href =  
    `tel:${encodedCode}`;
```

```
} catch (error) {
```

```
    console.error(  
        'Erreur lancement paiement :',  
        error  
    );
```

```
    showMessage(  
        'Impossible de lancer automatiquement  
le paiement. Utilisez le QR code.',  
        'error'  
    );  
}  
};
```

```
}
```

```
// -----
```

```
-----
```

```
// BOUTON "J'AI EFFECTUÉ LE PAIEMENT"
```

```
// -----
```

```
-----
```

```
const paymentDoneButton =  
  $('paymentDoneBtn');
```

```
if (paymentDoneButton) {
```

```
  paymentDoneButton.style.display =  
    '';
```

```
  paymentDoneButton.disabled =  
    false;
```

```
  paymentDoneButton.textContent =  
    "J'AI EFFECTUÉ LE PAIEMENT";
```



```
}
```

```
// -----
```

```
-----
```

```
// BOUTON HISTORIQUE
```

```
// -----
```

```
-----
```

```
if ($('#viewOrderBtn')) {
```

```
    $('#viewOrderBtn').style.display =  
    '';
```

```
    $('#viewOrderBtn').textContent =  
    'Voir ma commande';
```

```
}
```

```
// -----
```

```
-----
```

```
// PREUVE DE PAIEMENT (réinitialisation)
```

```
// -----
```

```
const proofLabelReset =  
  $('paymentPage')  
    ?.querySelector(  
      '.proof-upload-label'  
    );  
  
if (proofLabelReset) {  
  
  proofLabelReset.textContent =  
    'Preuve de paiement (capture d\'écran)';  
}  
  
resetProofUpload();  
  
const dropzoneReset =  
  $('proofDropzone');  
  
if (dropzoneReset) {  
  
  dropzoneReset.style.pointerEvents =
```

```
    ";

    dropzoneReset.style.opacity =
    ";
  }
}

function renderSellPaymentPage() {

  if (
    !currentOrder ||
    currentOrder.side !== 'sell'
  ) {
    return;
  }

  const networkName =
    CONFIG.networks[
      currentOrder.network
    ]?.name ||
    currentOrder.network ||
    '-';
```

```
// -----  
-----  
// DESCRIPTION  
// -----  
-----  
  
if ($('#paymentDescription')) {  
  
    $('#paymentDescription').textContent =  
        "Votre demande de vente est enregistrée.  
Envoyez vos USDT à l'adresse de dépôt  
indiquée.";  
}  
  
// -----  
-----  
// MONTANT  
// -----  
-----
```

```
if ($('#paymentAmountValue')) {  
  
    $('#paymentAmountValue').innerHTML =  
        `Vous recevrez :  
<strong>${formatNumber(currentOrder.receive  
Cfa)} FCFA</strong>`;   
}
```

```
// -----  
-----  
  
// TEXTE "SCANNEZ LE QR CODE" (masqué en  
vente)  
// -----  
-----
```

```
if ($('#qrInstructionText')) {  
  
    $('#qrInstructionText').style.display =  
        'none';  
}
```

```
// -----  
-----  
  
// QR BOX  
// Pour SELL, on n'utilise pas le QR Orange  
Money.  
// On affiche l'adresse NOA + l'option Binance.  
// -----  
-----  
  
const qrContainer =  
  $('orangeMoneyQr');  
  
if (qrContainer) {  
  
  qrContainer.innerHTML = `  
  
    <div class="payment-qr-text">  
  
      Envoyez exactement  
  
      <strong>
```

`${Number(currentOrder.netUsdt).toFixed(6)}`

USDT

`</strong>`

sur le réseau

`<strong>`

`${escapeHtml(networkName)}`

`</strong>.`

`</div>`

`<div`

`class="small break-word"`

`style="margin-top:12px;"`

`>`

`<strong>`

Adresse de dépôt NOA :

`</strong>`

`<br>`

```
    ${escapeHtml(
      currentOrder.walletAddress
    )}
```

```
</div>
```

```
<button
  type="button"
  id="copyDepositAddressBtn"
  class="copy-btn-inline"
>
```

```
   Copier l'adresse
```

```
</button>
```

```
<div class="payment-or-inline">
  OU
</div>
```

```
<div class="binance-transfer-box">
```

```
  <div class="binance-transfer-header">
```

```
    <span class="binance-logo">
```


B

Transférer via Binance

</div>

<div class="small">

ID Binance :

<strong id="binanceIdText">

`\${escapeHtml(CONFIG.binance.id)}`

</div>

<button

type="button"

id="copyBinanceIdBtn"

class="copy-btn-inline"

>



Copier l'ID Binance

</button>

</div>

```
`;  
`;
```

```
$('#copyDepositAddressBtn')  
  ?.addEventListener(  
    'click',  
    (event) => {  
  
      copyToClipboard(  
        currentOrder.walletAddress,  
        event.currentTarget  
      );  
    }  
  );
```

```
$('#copyBinanceldBtn')  
  ?.addEventListener(  
    'click',  
    (event) => {  
  
      copyToClipboard(  

```

```
        CONFIG.binance.id,  
        event.currentTarget  
    );  
    }  
    );  
}  
  
// -----  
-----  
// BOUTON ORANGE MONEY  
// -----  
-----  
  
const payButton =  
    $('orangeMoneyPayBtn');  
  
if (payButton) {  
  
    payButton.style.display =  
        'none';  
}
```

```
payButton.onclick =  
    null;  
}  
  
// -----  
-----  
  
// "OU" (statique, masqué — on utilise celui  
généré ci-dessus)  
// -----  
-----  
  
const paymentOr =  
    $('paymentPage')  
    ?.querySelector(  
        'payment-or'  
    );  
  
if (paymentOr) {  
  
    paymentOr.style.display =  
        'none';  
}
```

```
}
```

```
// -----
```

```
-----
```

```
// BOUTON "J'AI ENVOYÉ LES USDT"
```

```
// -----
```

```
-----
```

```
if ($('#paymentDoneBtn')) {
```

```
    $('#paymentDoneBtn').style.display =  
    '';
```

```
    $('#paymentDoneBtn').disabled =  
    false;
```

```
    $('#paymentDoneBtn').textContent =  
    "J'AI ENVOYÉ LES USDT";
```

```
}
```

```
// -----
```

```
-----  
// LIBELLÉ PREUVE  
// -----  
-----  
  
const proofLabel =  
  $('paymentPage')  
    ?.querySelector(  
      '.proof-upload-label'  
    );  
  
if (proofLabel) {  
  
  proofLabel.textContent =  
    "Preuve d'envoi des USDT (capture  
d'écran)";  
}  
  
resetProofUpload();  
  
const dropzoneReset =  
  $('proofDropzone');
```

```
if (dropzoneReset) {  
  
    dropzoneReset.style.pointerEvents =  
        '';  
  
    dropzoneReset.style.opacity =  
        '';  
}  
  
// -----  
-----  
// SÉCURITÉ  
// -----  
-----  
  
const securityBox =  
    $('paymentPage')  
        ?.querySelector(  
            'payment-security-box'  
        );
```

```
if (securityBox) {  
  
    securityBox.innerHTML = `  
  
         Vérifiez attentivement le réseau  
  
        <strong>  
            ${escapeHtml(networkName)}  
        </strong>  
        avant d'envoyer vos USDT.  
  
    `;  
}
```

```
// -----  
-----  
// AVERTISSEMENT  
// -----  
-----
```

```
const warning =  
    $('paymentPage')
```



```
?.querySelector(  
  '.warning-box'  
);
```

```
if (warning) {
```

```
  warning.innerHTML = `
```

```
    ⚠ <strong>IMPORTANT :</strong>
```

```
    <br><br>
```

```
    Envoyez exactement
```

```
    <strong>
```

```
    ${Number(currentOrder.netUsdt).toFixed(6)}
```

```
    USDT
```

```
    </strong>
```

```
    sur le réseau
```

```
<strong>
    ${escapeHtml(networkName)}
</strong>.
```

```
<br><br>
```

Adresse de dépôt NOA :

```
<div
    class="break-word"
    style="margin-top:8px;"
>
```

```
<strong>
    ${escapeHtml(
        currentOrder.walletAddress
    )}
</strong>
```

```
</div>
```

```
<br>
```

Une erreur de réseau ou d'adresse peut
entraîner
une perte des fonds.

Après réception et vérification de vos
USDT,
nous vous enverrons

`$(formatNumber(currentOrder.receiveCfa))`
FCFA

sur votre Orange Money.

;
}

// -----

```
-----  
// BOUTON HISTORIQUE  
// -----  
-----  
  
if ($('#viewOrderBtn')) {  
  
    $('#viewOrderBtn').style.display =  
        '';  
  
    $('#viewOrderBtn').textContent =  
        'Voir ma commande';  
}  
}  
  
  
//  
-----  
-----  
// DÉCLARER PAIEMENT  
// RPC SUPABASE SÉCURISÉ  
//  
-----
```

```
=====

//
=====

=====

// PREUVE DE PAIEMENT (upload)
//
=====

=====

const PROOF_MAX_SIZE_BYTES =
    5 * 1024 * 1024;

const PROOF_ALLOWED_TYPES = [

    'image/jpeg',

    'image/png',

    'image/webp'

];
```

```
function resetProofUpload() {  
  
    selectedProofFile =  
        null;  
  
    const fileInput =  
        $('proofFileInput');  
  
    if (fileInput) {  
        fileInput.value =  
            '';  
    }  
  
    const content =  
        $('proofDropzoneContent');  
  
    if (content) {  
        content.classList.remove(  
            'hidden')
```

```
);  
}
```

```
const previewWrap =  
  $('proofPreviewWrap');
```

```
if (previewWrap) {  
  previewWrap.classList.add(  
    'hidden'  
  );  
}
```

```
const previewImg =  
  $('proofPreviewImg');
```

```
if (previewImg) {  
  previewImg.src =  
    '';  
}
```

```
const status =  
  $('proofUploadStatus');  
  
if (status) {  
  status.textContent =  
    '';  
  status.className =  
    '';  
}  
  
const dropzone =  
  $('proofDropzone');  
  
if (dropzone) {  
  dropzone.classList.remove(  
    'dragover'  
  );  
}  
}  
  
function handleProofFile(
```



```
file
){

    const status =
        $('proofUploadStatus');

    if (!file) {
        return;
    }

    if (
        !PROOF_ALLOWED_TYPES.includes(
            file.type
        )
    ){

        if (status) {
            status.textContent =
                'Format non supporté. Utilisez une image
                JPG, PNG ou WEBP';
            status.className =
```

```
        'error';
    }

    return;
}

if (
    file.size >
    PROOF_MAX_SIZE_BYTES
){

    if (status) {
        status.textContent =
            'Fichier trop volumineux (5 Mo maximum).';
        status.className =
            'error';
    }

    return;
}
```

```
selectedProofFile =  
  file;  
  
if (status) {  
  status.textContent =  
    ”;  
  status.className =  
    ”;  
}  
  
const reader =  
  new FileReader();  
  
reader.onload =  
  () => {  
  
    const previewImg =  
      $('proofPreviewImg');  
  
    if (previewImg) {
```

```
    previewImg.src =  
      reader.result;  
  }
```

```
const content =  
  $('proofDropzoneContent');
```

```
if (content) {  
  content.classList.add(  
    'hidden'  
  );  
}
```

```
const previewWrap =  
  $('proofPreviewWrap');
```

```
if (previewWrap) {  
  previewWrap.classList.remove(  
    'hidden'  
  );  
}
```

```
};
```

```
    reader.readAsDataURL(  
      file  
    );  
  }  
}
```

```
function initProofUploadEvents() {
```

```
  const dropzone =  
    $('proofDropzone');
```

```
  const fileInput =  
    $('proofFileInput');
```

```
  const removeBtn =  
    $('proofRemoveBtn');
```

```
  if (dropzone) {
```

```
dropzone.addEventListener(  
  'click',  
  event => {  
  
    if (  
      event.target.closest(  
        '#proofRemoveBtn'  
      )  
    ){  
      return;  
    }  
  
    fileInput?.click();  
  }  
);
```

```
dropzone.addEventListener(  
  'dragover',  
  event => {  
  
    event.preventDefault();
```

```
dropzone.classList.add(
  'dragover'
);
}
```

```
dropzone.addEventListener(
  'dragleave',
  () => {
```

```
    dropzone.classList.remove(
      'dragover'
    );
  }
);
```

```
dropzone.addEventListener(
  'drop',
  event => {
```

```
    event.preventDefault();
```

```
dropzone.classList.remove(  
  'dragover'  
);
```

```
const file =  
  event.dataTransfer  
    ?.files?.[0];
```

```
if (file) {  
  handleProofFile(  
    file  
  );  
}  
}  
);  
}
```

```
fileInput?.addEventListener(  
  'change',  
  event => {
```



```
const file =  
  event.target  
    .files?.[0];  
  
if (file) {  
  handleProofFile(  
    file  
  );  
}  
}  
);  
  
removeBtn?.addEventListener(  
  'click',  
  event => {  
  
    event.stopPropagation();  
  
    resetProofUpload();  
  }  
);
```

```
}
```

```
async function uploadProofFile(  
  file,  
  orderId  
) {
```

```
  const extension =
```

```
  (  
    file.name  
      .split('.')  
      .pop() ||  
    'jpg'  
  )
```

```
  .toLowerCase();
```

```
  const path =
```

```
`${currentUser.id}/${orderId}-${Date.now()}.${ext  
ension}`;
```

```
const {
  error: uploadError
} =
  await supabaseClient
    .storage
    .from('payment-proofs')
    .upload(
      path,
      file,
      {
        contentType:
          file.type,

        upsert:
          false
      }
    );

if (uploadError) {
  throw uploadError;
}
```

```
    return path;  
  }
```

```
  async function declarePayment() {
```

```
    hideMessage();
```

```
    if (  
      !currentUser  
    ){
```

```
      return showMessage(  
        'Votre session a expiré. Veuillez vous  
reconnecter.',  
        'error'  
      );  
    }
```

```
if (
  !currentOrder?.id
){

  return showMessage(
    'Commande introuvable.',
    'error'
  );
}
```

```
const isSell =
  currentOrder.side ===
  'sell';
```

```
if (!selectedProofFile) {

  return showMessage(
    isSell
      ? "Veuillez joindre une capture d'écran de
votre envoi de USDT avant de continuer."
      : "Veuillez joindre une capture d'écran de
```

```
    votre paiement avant de continuer.”,  
    'error'  
  );  
}
```

```
const button =  
  $('paymentDoneBtn');
```

```
const originalText =  
  button?.textContent;
```

```
if (button) {  
  
  button.disabled =  
    true;  
  
  button.textContent =  
    'Confirmation...';  
}
```

```
const status =  
  $('proofUploadStatus');  
  
try {  
  
  // -----  
-----  
  // Vérifier que la session est toujours active  
  // -----  
-----  
  
  const {  
    data: sessionData,  
    error: sessionError  
  } =  
    await supabaseClient.auth  
      .getSession();  
  
  if (sessionError) {  
    throw sessionError;  
  }  
}
```

```
}
```

```
if (
```

```
  !sessionData?.session?.user
```

```
) {
```

```
  throw new Error(
```

```
    "Votre session n'est plus active."
```

```
  );
```

```
}
```

```
currentUser =
```

```
  sessionData.session.user;
```

```
// -----
```

```
-----
```

```
  // ENVOI DE LA PREUVE DE PAIEMENT (bucket  
privé)
```

```
// -----
```

```
-----
```



```
if (status) {  
  status.textContent =  
    'Envoi de la preuve de paiement...';  
  status.className =  
    '';  
}
```

```
const proofPath =  
  await uploadProofFile(  
    selectedProofFile,  
    currentOrder.id  
  );
```

```
if (status) {  
  status.textContent =  
    'Preuve envoyée ✓';  
  status.className =  
    'success';  
}
```

```
// -----  
-----  
  
// APPEL DE LA FONCTION SQL SÉCURISÉE  
// Aucun UPDATE direct depuis le navigateur.  
// -----  
-----  
  
const {  
  data,  
  error  
} =  
await supabaseClient.rpc(  
  'declare_order_payment',  
  {  
    p_order_id:  
      currentOrder.id,  
  
    p_payment_proof_url:  
      proofPath  
  }  
);
```

```
if (error) {  
  throw error;  
}
```

```
if (!data) {  
  
  throw new Error(  
    "La commande n'a pas pu être mise à  
jour."  
  );  
}
```

```
// -----  
-----  
  
// Supabase peut retourner un objet ou un  
tableau selon  
  
// le contexte de la fonction RPC.  
  
// -----  
-----
```

```
const updatedOrder =
  Array.isArray(data)
    ? data[0]
    : data;

if (!updatedOrder) {

  throw new Error(
    "La commande mise à jour n'a pas été
retournée."
  );
}

currentOrder.status =
  updatedOrder.status ||
  'payment_declared';

currentOrder.paymentStatus =
  updatedOrder.payment_status ||
  'declared';
```

```
showMessage(  
    isSell  
        ? 'Envoi déclaré. Votre commande est  
maintenant en attente de vérification.'  
        : 'Paiement déclaré. Votre commande est  
maintenant en attente de vérification.',  
    'success'  
);  
  
// Le bouton ne doit plus être cliqué  
inutilement.  
if (button) {  
  
    button.disabled =  
        true;  
  
    button.textContent =  
        isSell  
            ? 'ENVOI DÉCLARÉ ✓'  
            : 'PAIEMENT DÉCLARÉ ✓';  
}
```

```
}
```

```
const dropzoneAfterSuccess =  
  $('proofDropzone');
```

```
if (dropzoneAfterSuccess) {
```

```
  dropzoneAfterSuccess.style.pointerEvents  
=  
  'none';
```

```
  dropzoneAfterSuccess.style.opacity =  
    '0.6';  
}
```

```
await loadOrderHistory();
```

```
} catch (error) {
```

```
  console.error(
```

```
'ERREUR DÉCLARATION PAIEMENT :',  
  error  
);
```

```
showMessage(  
  'Impossible de confirmer le paiement : ' +  
  getSupabaseErrorMessage(error),  
  'error'  
);
```

```
if (status) {  
  
  status.textContent =  
    "Échec de l'envoi. Réessayez."  
  
  status.className =  
    'error';  
}
```

```
if (button) {
```

```
button.disabled =  
    false;  
  
button.textContent =  
    originalText ||  
    (isSell  
        ? "J'AI ENVOYÉ LES USDT"  
        : "J'AI EFFECTUÉ LE PAIEMENT");  
    }  
    }  
}
```

```
//  
=====
```

=====

```
// HISTORIQUE  
//  
=====
```

=====

```
async function loadOrderHistory() {
```



```
const list =  
  $('ordersList');  
  
if (!list) {  
  return;  
}  
  
if (!currentUser) {  
  
  list.innerHTML =  
    '<div class="small center">Connectez-vous  
pour voir vos commandes.</div>';  
  
  return;  
}  
  
list.innerHTML =  
  '<div class="small center">Chargement des  
commandes...</div>';
```

```
try {  
  
  const {  
    data,  
    error  
  } =  
    await supabaseClient  
      .from('orders')  
      .select("")  
      .eq(  
        'user_id',  
        currentUser.id  
      )  
      .order(  
        'created_at',  
        {  
          ascending: false  
        }  
      );
```

```
if (error) {  
  throw error;  
}
```

```
if (  
  !data ||  
  data.length === 0  
) {
```

```
  list.innerHTML =  
    '<div class="small center">Vous n\'avez  
encore aucune commande.</div>';
```

```
  return;  
}
```

```
list.innerHTML =  
  data  
    .map(  
      renderOrderCard  
    )
```

```
.join(");
```

```
} catch (error) {
```

```
  console.error(  
    'ERREUR HISTORIQUE :',  
    error  
  );
```

```
list.innerHTML = `
```

```
<div class="small center">
```

```
  Impossible de charger vos commandes.
```

```
<br>
```

```
    ${escapeHtml(  
      getSupabaseErrorMessage(error)  
    )}
```

```
</div>
```

```
    ;  
  }  
}
```

```
//
```

```
=====
```

```
=====
```

```
// CARTE COMMANDE
```

```
//
```

```
=====
```

```
=====
```

```
function renderOrderCard(  
  order
```

```
){
```

```
{
```

```
  const side =
```

```
    order.side;
```

```
const type =  
  side === 'buy'  
    ? 'Achat USDT'  
    : 'Vente USDT';
```

```
const status =  
  order.status ||  
  'pending';
```

```
const statusLabel =  
  getStatusLabel(  
    status  
  );
```

```
const networkName =  
  CONFIG.networks[  
    order.network  
  ]?.name ||  
  order.network ||  
  '-';
```

```
if (
  side === 'buy'
){

  return `

    <div class="order-card">

      <div class="order-header">

        <div class="order-type">
          ${escapeHtml(type)}
        </div>

        <span class="status
${escapeHtml(status)}">
          ${escapeHtml(statusLabel)}
        </span>

      </div>
```

```
<div class="order-row">  
  <span>Montant payé</span>  
  <strong>  
    ${formatNumber(order.amount_cfa)}  
FCFA  
  </strong>  
</div>
```

```
<div class="order-row">  
  <span>USDT acheté</span>  
  <strong>  
    ${Number(order.usdt_amount ||  
0).toFixed(6)} USDT  
  </strong>  
</div>
```

```
<div class="order-row">  
  <span>Frais</span>  
  <strong>  
    ${Number(order.fee_usdt || 0)} USDT  
  </strong>  
</div>
```



```
<div class="order-row">
  <span>USDT net reçu</span>
  <strong>
    ${Number(order.net_usdt ||
0).toFixed(6)} USDT
  </strong>
</div>

<div class="order-row">
  <span>Réseau</span>
  <strong>
    ${escapeHtml(networkName)}
  </strong>
</div>

<div class="order-row">
  <span>Date</span>
  <strong>
    ${formatDate(order.created_at)}
  </strong>
</div>

</div>
```

```
;
}
```

```
return`
```

```
<div class="order-card">
```

```
<div class="order-header">
```

```
<div class="order-type">
```

```
  ${escapeHtml(type)}
```

```
</div>
```

```
<span class="status
```

```
${escapeHtml(status)}">
```

```
  ${escapeHtml(statusLabel)}
```

```
</span>
```

```
</div>
```

```
<div class="order-row">
```

```
<span>USDT vendu</span>
<strong>
  ${Number(order.usdt_amount ||
0).toFixed(6)} USDT
</strong>
</div>

<div class="order-row">
  <span>Frais réseau</span>
  <strong>
    ${Number(order.fee_usdt || 0).toFixed(6)}
    USDT
  </strong>
</div>

<div class="order-row">
  <span>USDT net</span>
  <strong>
    ${Number(order.net_usdt || 0).toFixed(6)}
    USDT
  </strong>
</div>
```

```
<div class="order-row">
  <span>Montant reçu</span>
  <strong>
    ${formatNumber(order.receive_cfa || 0)}
    FCFA
  </strong>
</div>
```

```
<div class="order-row">
  <span>Réseau</span>
  <strong>
    ${escapeHtml(networkName)}
  </strong>
</div>
```

```
<div class="order-row">
  <span>Date</span>
  <strong>
    ${formatDate(order.created_at)}
  </strong>
</div>
```

```
</div>
```

```
;
}
```

```
//
```

```
=====
```

```
=====
```

```
// STATUTS
```

```
//
```

```
=====
```

```
=====
```

```
function getStatusLabel(
  status
){
```

```
  const labels = {
```

```
    pending:
      'En attente',
```

```
    payment_declared:
```

'Paiement déclaré',

processing:

'En traitement',

completed:

'Terminée',

cancelled:

'Annulée',

rejected:

'Refusée'

};

return (

labels[status] ||

status ||

'En attente'

);

}

```
//  
=====
```

=====

```
// SUPPORT  
//  
=====
```

=====

```
async function loadOrdersForDispute() {  
  
  const select =  
    $('disputeOrder');  
  
  if (  
    !select ||  
    !currentUser  
  ) {  
    return;  
  }  
}
```

```
try {  
  
  const {  
    data,  
    error  
  } =  
    await supabaseClient  
      .from('orders')  
      .select(  
        'id,side,created_at,status'  
      )  
      .eq(  
        'user_id',  
        currentUser.id  
      )  
      .order(  
        'created_at',  
        {  
          ascending: false  
        }  
      );
```



```
if (error) {  
  throw error;  
}
```

```
select.innerHTML =  
  '<option value="">Sélectionner une  
commande</option>';
```

```
(data || []).  
  .forEach(  
    order => {
```

```
      const type =  
        order.side === 'buy'  
        ? 'Achat'  
        : 'Vente';
```

```
      const option =  
        document.createElement(
```

```
        'option'  
    );  
  
    option.value =  
        order.id;  
  
    option.textContent =  
        `${type} -  
        ${formatDate(order.created_at)} -  
        ${getStatusLabel(order.status)}`;  
  
    select.appendChild(  
        option  
    );  
  
    }  
    );  
  
    } catch (error) {
```

```
    console.error(  
      'Erreur commandes support :',  
      error  
    );  
  }  
}
```

```
//
```

```
=====
```

```
=====
```

```
// ENVOYER LITIGE
```

```
//
```

```
=====
```

```
=====
```

```
async function submitDispute(  
  event
```

```
) {
```

```
  event.preventDefault();
```

```
hideMessage();
```

```
if (!currentUser) {
```

```
    return showMessage(  
        'Vous devez être connecté.',  
        'error'  
    );  
}
```

```
const orderId =  
    $('disputeOrder')  
        ?.value || '';
```

```
const subject =  
    $('disputeSubject')  
        ?.value  
        ?.trim() || '';
```

```
const message =  
  `${disputeMessage}`  
    ?.value  
    ?.trim() || ` `;  
  
if (!orderId) {  
  
  return showMessage(  
    'Veuillez sélectionner une commande.',  
    'error'  
  );  
}  
  
if (!subject) {  
  
  return showMessage(  
    'Veuillez indiquer le sujet du problème.',  
    'error'  
  );  
}
```

```
if (!message) {  
  
    return showMessage(  
        'Veuillez expliquer votre problème.',  
        'error'  
    );  
}
```

```
const button =  
    event.submitter ||  
    $('disputeForm')  
        ?.querySelector(  
            'button[type="submit"]'  
        );
```

```
const originalText =  
    button?.textContent;
```

```
if (button) {
```

```
button.disabled =  
  true;  
  
button.textContent =  
  'Envoi...';  
}
```

```
try {  
  
  const payload = {  
  
    user_id:  
      currentUser.id,  
  
    order_id:  
      orderId,  
  
    subject:  
      subject,  
  
    message:
```

```
    message,  
  
    status:  
      'open'  
  
  };  
  
  const {  
    error  
  } =  
    await supabaseClient  
      .from('disputes')  
      .insert(  
        payload  
      );  
  
  if (error) {  
    throw error;  
  }
```



```
$('#disputeForm')
```

```
?.reset();
```

```
showMessage(
```

```
  'Votre demande a été envoyée à notre  
équipe.',
```

```
  'success'
```

```
);
```

```
await loadDisputes();
```

```
} catch (error) {
```

```
  console.error(
```

```
    'Erreur création litige :',
```

```
    error
```

```
  );
```

```
showMessage(
```

```
        "Impossible d'envoyer votre demande : " +  
        getSupabaseErrorMessage(error),  
        'error'  
    );  
  
    } finally {  
  
        if (button) {  
  
            button.disabled =  
                false;  
  
            button.textContent =  
                originalText ||  
                'Envoyer la demande';  
        }  
    }  
  
    //  
    =====  
    =====
```

```
// CHARGER LITIGES
//
=====

async function loadDisputes() {

  const list =
    $('disputesList');

  if (
    !list ||
    !currentUser
  ) {
    return;
  }

  list.innerHTML =
    '<div class="small center">Chargement...
</div>';
```

```
try {  
  
  const {  
    data,  
    error  
  } =  
    await supabaseClient  
      .from('disputes')  
      .select("")  
      .eq(  
        'user_id',  
        currentUser.id  
      )  
      .order(  
        'created_at',  
        {  
          ascending: false  
        }  
      );  
  
  if (error) {
```

```
    throw error;  
  }
```

```
  if (  
    !data ||  
    data.length === 0  
  ) {
```

```
    list.innerHTML =  
      '<div class="small center">Aucune  
demande pour le moment.</div>';
```

```
    return;  
  }
```

```
  list.innerHTML =  
    data  
      .map(  
        renderDisputeCard  
      )  
      .join("");
```

```
} catch (error) {  
  
    console.error(  
        'Erreur chargement litiges :',  
        error  
    );  
  
    list.innerHTML =  
        '<div class="small center">Impossible de  
charger les demandes.</div>';  
    }  
}  
  
//  
=====
```

=====

```
// CARTE LITIGE  
//  
=====
```

```
=====
```

```
function renderDisputeCard(  
  dispute  
) {
```

```
  const status =  
    dispute.status ||  
    'open';
```

```
  const statusLabel =  
    status === 'open'  
    ? 'Ouvert'  
    : status === 'closed'  
    ? 'Fermé'  
    : status;
```

```
  return `
```

```
    <div class="order-card">
```

```
<div class="order-header">
```

```
  <strong>
```

```
    ${escapeHtml(  
      dispute.subject ||  
      'Demande'  
    )}
```

```
  </strong>
```

```
  <span class="status
```

```
    ${escapeHtml(status)}">
```

```
    ${escapeHtml(statusLabel)}
```

```
  </span>
```

```
</div>
```

```
<p>
```

```
  ${escapeHtml(  
    dispute.message ||  
    "
```

```
  )}
```

```
</p>
```



```
<div class="small">
  ${formatDate(
    dispute.created_at
  )}
</div>
```

```
</div>
```

```
;
}
```

```
//
```

```
=====
```

```
=====
```

```
// PROFIL
```

```
//
```

```
=====
```

```
=====
```

```
async function saveProfile(
  event
){
```

```
event?.preventDefault();
```

```
hideMessage();
```

```
if (!currentUser) {
```

```
    return showMessage(  
        'Vous devez être connecté.',  
        'error'  
    );
```

```
}
```

```
const name =
```

```
    $('profileName')  
        ?.value  
        ?.trim() || '';
```

```
const phone =
```

```
    normalizePhone(  
        name,
```

```
    $('profilePhone')  
      ?.value  
  );
```

```
const country =  
  $('profileCountry')  
    ?.value  
    ?.trim() ||  
  'Burkina Faso';
```

```
if (!name) {  
  
  return showMessage(  
    'Veuillez saisir votre nom.',  
    'error'  
  );  
}
```

```
if (!phone) {
```

```
return showMessage(  
    'Veuillez saisir votre numéro.',  
    'error'  
);  
}  
  
if (  
    /^[+226\d{8}$]/.test(  
        phone  
    )  
) {  
  
    return showMessage(  
        'Veuillez saisir un numéro du Burkina Faso  
valide de 8 chiffres.',  
        'error'  
    );  
}  
  
try {
```

```
const {
  data,
  error
} =
  await supabaseClient
    .from('profiles')
    .upsert(

      {

        id:
          currentUser.id,

        full_name:
          name,

        phone:
          phone,

        country:
          country

      },
```

```
        {
            onConflict:
                'id'
        }

    )
    .select(
        'id,full_name,phone,country'
    )
    .maybeSingle();

    if (error) {
        throw error;
    }

    currentProfile =
        data || {

            id:
                currentUser.id,
```

```
full_name:
```

```
  name,
```

```
phone:
```

```
  phone,
```

```
country:
```

```
  country
```

```
};
```

```
await supabaseClient.auth
```

```
.updateUser({
```

```
  data: {
```

```
    full_name:
```

```
      name,
```

```
    phone:
```

```
      phone,
```

```
country:
```

```
country
```

```
}
```

```
});
```

```
updateUserInterface();
```

```
showMessage(
```

```
  'Profil mis à jour avec succès.',
```

```
  'success'
```

```
);
```

```
} catch (error) {
```

```
  console.error(
```

```
    'Erreur sauvegarde profil :',
```

```
    error
```



```
);
```

```
showMessage(  
  'Impossible de sauvegarder le profil : ' +  
  getSupabaseErrorMessage(error),  
  'error'  
);  
}  
}
```

```
//
```

```
=====
```

```
=====
```

```
// PARRAINAGE
```

```
//
```

```
=====
```

```
=====
```

```
function copyReferralCode() {
```

```
  const code =
```

```
$(`referralCodeDisplay`)  
?.value;  
  
if (  
  !code ||  
  code === 'Génération en cours...'  
) {  
  return;  
}  
  
navigator.clipboard  
?.writeText(code)  
.then(() => {  
  
  showMessage(  
    'Code de parrainage copié !',  
    'success'  
  );  
  
})  
.catch(() => {  
  
  showMessage(  

```

```
        'Impossible de copier le code.',  
        'error'  
    );  
  
    });  
}
```

```
function toggleWithdrawalFields() {
```

```
    const method =  
        $('withdrawalMethod')  
        ?.value;
```

```
    if (method === 'usdt') {
```

```
        $('withdrawalPhoneField')  
        ?.classList  
        .add('hidden');
```

```
        $('withdrawalWalletField')  
        ?.classList  
        .remove('hidden');
```

```
    } else {

        $('withdrawalPhoneField')
            ?.classList
            .remove('hidden');

        $('withdrawalWalletField')
            ?.classList
            .add('hidden');

    }
}

async function submitReferralWithdrawal() {

    const amount =
        parseFloat(
            $('withdrawalAmount')
                ?.value
        );
}
```

```
const method =  
  $('withdrawalMethod')  
    ?.value ||  
  'cash';
```

```
const walletAddress =  
  $('withdrawalWallet')  
    ?.value  
    .trim() ||  
  null;
```

```
const payoutPhone =  
  $('withdrawalPhone')  
    ?.value  
    .trim() ||  
  null;
```

```
if (  
  !amount ||  
  amount < 5000  
) {
```

```
return showMessage(  
    'Le montant minimum de retrait est de  
5000 FCFA.',  
    'error'  
);  
}  
  
if (  
    method === 'usdt' &&  
    !walletAddress  
) {  
  
    return showMessage(  
        'Merci de renseigner votre adresse wallet  
USDT.',  
        'error'  
    );  
}  
  
if (  
    method === 'cash' &&
```

```
!payoutPhone
){

    return showMessage(
        'Merci de renseigner votre numéro Mobile
Money.',
        'error'
    );
}
```

```
const button =
    $('submitWithdrawalBtn');
```

```
const original =
    button?.textContent;
```

```
if (button) {

    button.disabled = true;
    button.textContent = 'Envoi en cours...';
}
```

```
try {

  const {
    data,
    error
  } =
    await supabaseClient
      .rpc(
        'request_referral_withdrawal',
        {
          p_amount: amount,
          p_method: method,
          p_wallet_address: walletAddress,
          p_payout_phone: payoutPhone
        }
      );

  if (error) {
    throw error;
  }
}
```



```
if (
  data &&
  data.success === false
){

  return showMessage(
    data.error ||
    'Impossible de traiter la demande.',
    'error'
  );
}

showMessage(
  'Demande de retrait envoyée avec succès.',
  'success'
);

$('withdrawalForm')
  ?.classList
  .add('hidden');
```

```
if ($('#withdrawalAmount')) {  
  $('#withdrawalAmount').value = "";  
}  
  
await loadUserProfile();  
  
} catch (error) {  
  
  console.error(  
    'Erreur demande de retrait :',  
    error  
  );  
  
  showMessage(  
    'Erreur : ' +  
    getSupabaseErrorMessage(error),  
    'error'  
  );  
  
} finally {
```

```
    if (button) {  
  
        button.disabled = false;  
        button.textContent = original;  
    }  
}  
}  
  
//  
=====
```

=====

```
// MOT DE PASSE  
//  
=====
```

=====

```
async function changePassword(  
    event  
) {  
  
    event?.preventDefault();
```

```
hideMessage());
```

```
const password =  
  $('newPassword')  
    ?.value || '';
```

```
const confirmation =  
  $('confirmPassword')  
    ?.value || '';
```

```
if (  
  password.length <  
    6  
) {
```

```
  return showMessage(  
    'Le nouveau mot de passe doit contenir au  
    moins 6 caractères.',  
    'error'  
  );
```

```
}
```

```
if (
```

```
  password !==
```

```
  confirmation
```

```
) {
```

```
  return showMessage(
```

```
    'Les mots de passe ne correspondent pas.',
```

```
    'error'
```

```
  );
```

```
}
```

```
try {
```

```
  const {
```

```
    error
```

```
  } =
```

```
    await supabaseClient.auth
```

```
      .updateUser({
```

password

 $\})$;

```
if (error) {
    throw error;
}
```

```
$( 'passwordForm' )
  ?.reset();
```

```
showMessage(  
    'Mot de passe modifié avec succès.',  
    'success'  
);
```

```
} catch (error) {
```

```
console.error(
```

```
    'Erreur mot de passe :',  
    error  
  );
```

```
  showMessage(  
    'Impossible de modifier le mot de passe : ' +  
    getSupabaseErrorMessage(error),  
    'error'  
  );  
}  
}
```

```
//  
=====
```

=====

```
// ÉVÉNEMENTS  
//  
=====
```

=====

```
function setupEvents() {
```

```
$('#loginForm')  
  ?.addEventListener(  
    'submit',  
    loginUser  
  );
```

```
$('#registerForm')  
  ?.addEventListener(  
    'submit',  
    registerUser  
  );
```

```
$('#forgotPasswordForm')  
  ?.addEventListener(  
    'submit',  
    sendPasswordReset  
  );
```

```
$('#forgotPasswordLink')
```



```
?.addEventListener(  
  'click',  
  showForgotPasswordForm  
);
```

```
$('#backToLoginBtn')  
?.addEventListener(  
  'click',  
  showLoginForm  
);
```

```
initPasswordToggles();
```

```
initWhatsAppButton();
```

```
$('#logoutBtn')  
?.addEventListener(  
  'click',  
  logoutUser  
);
```

```
);
```

```
$('#loginTab')  
  ?.addEventListener(  
    'click',  
    showLoginForm  
  );
```

```
$('#registerTab')  
  ?.addEventListener(  
    'click',  
    showRegisterForm  
  );
```

```
$('#buyTab')  
  ?.addEventListener(  
    'click',  
    setBuyMode  
  );
```

```
$('#sellTab')  
  ?.addEventListener(  
    'click',  
    setSellMode  
  );
```

```
$('#trc20Option')  
  ?.addEventListener(  
    'click',  
    () => {  
  
      selectNetwork(  
        'trc20'  
      );  
  
    }  
  );
```

```
$('#bp20Option')  
  ?.addEventListener(  
    'click',  
    () => {  
  
      selectNetwork(  
        'bp20'  
      );  
  
    }  
  );
```

```
      'click',  
      () => {  
  
        selectNetwork(  
          'bp20'  
        );  
  
      }  
    );  
  
    $('amountInput')  
      ?.addEventListener(  
        'input',  
        updateCalculator  
      );  
  
    document.addEventListener(  
      'input',  
      event => {  
  
        if (
```

```
        event.target?.id ===  
        'sellPayoutPhone'  
    ){  
  
        event.target.value =  
            event.target.value  
                .replace(  
                    /\D/g,  
                    ''  
                )  
                .slice(  
                    0,  
                    8  
                );  
    }  
  
    }  
);  
  
$(`reviewOrderBtn`)  
    ?.addEventListener(  
        'click',
```

```
reviewOrder  
);
```

```
$('#placeOrderBtn')  
?.addEventListener(  
  'click',  
  placeOrder  
);
```

```
$('#cancelReviewBtn')  
?.addEventListener(  
  'click',  
  cancelReview  
);
```

```
$('#paymentDoneBtn')  
?.addEventListener(  
  'click',  
  declarePayment  
);
```

```
initProofUploadEvents();
```

```
$('#viewOrderBtn')  
  ?.addEventListener(  
    'click',  
    () => {  
  
      showSubPage(  
        'ordersPage'  
      );  
  
    }  
  );
```

```
$('#profileForm')  
  ?.addEventListener(  
    'submit',  
    saveProfile  
  );
```

```
$('#copyReferralCodeBtn')  
  ?.addEventListener(  
    'click',  
    copyReferralCode  
  );
```

```
$('#openWithdrawalBtn')  
  ?.addEventListener(  
    'click',  
    () => {
```

```
      $('#withdrawalForm')  
        ?.classList  
        .toggle('hidden');
```

```
    }  
  );
```

```
$('#withdrawalMethod')
```



```
?.addEventListener(  
  'change',  
  toggleWithdrawalFields  
);
```

```
$('#submitWithdrawalBtn')  
?.addEventListener(  
  'click',  
  submitReferralWithdrawal  
);
```

```
$('#passwordForm')  
?.addEventListener(  
  'submit',  
  changePassword  
);
```

```
$('#disputeForm')  
?.addEventListener(  
  'submit',
```

```
submitDispute  
);
```

```
document  
  .querySelectorAll(  
    '.nav-btn'  
  )  
  .forEach(  
    button => {  
  
      button.addEventListener(  
        'click',  
        () => {  
  
          const page =  
            button.dataset.page;  
  
          if (page) {  
  
            showSubPage(  
              page
```

```
    );  
  }  
  
  }  
);  
  
}  
);  
  
$(startBtn)  
?.addEventListener(  
  'click',  
  () => {  
  
    showSubPage(  
      'exchangePage'  
    );  
  
  }  
);
```

```
document
  .querySelectorAll(
    '[data-open-exchange]'
  )
  .forEach(
    button => {

      button.addEventListener(
        'click',
        () => {

          showSubPage(
            'exchangePage'
          );

        }
      );

    }
  );
```

```
document
```

```
.querySelectorAll(  
  '[data-home]'  
)  
.forEach(  
  button => {  
  
    button.addEventListener(  
      'click',  
      () => {  
  
        showSubPage(  
          'homePage'  
        );  
  
      }  
    );  
  
  }  
);  
  
}  
  
//
```

```
=====

=====

// AUTH LISTENER
//

=====

=====

function setupAuthListener() {

  if (
    authListenerReady
  ) {
    return;
  }

  authListenerReady =
    true;

  supabaseClient.auth
    .onAuthStateChange(
      (
```

```
    event,  
    session  
  ) => {  
  
    currentUser =  
      session?.user ||  
      null;  
  
    if (  
      event ===  
        'SIGNED_OUT' ||  
      !currentUser  
    ) {  
  
      currentProfile =  
        null;  
  
      currentOrder =  
        null;  
  
      showAuthPage();  
    }  
  }  
}
```

```
        showLoginForm();

        return;
    }

    };
}

//
=====

=====
// DÉMARRAGE
//
=====

=====

async function boot() {

    try {

        setupEvents();
```



```
setupAuthListener();
```

```
const {  
  data,  
  error  
} =  
  await supabaseClient.auth  
    .getSession();
```

```
if (error) {  
  
  console.warn(  
    'Erreur récupération session :',  
    error  
  );  
}
```

```
currentUser =  
  data?.session?.user ||
```

```
    null;
```

```
    if (currentUser) {
```

```
        await initializeApplication();
```

```
    } else {
```

```
        showAuthPage();
```

```
        showLoginForm();
```

```
    }
```

```
    setBuyMode();
```

```
    selectNetwork(
```

```
        'trc20'
```

```
    );
```

```
    updateRatesUI();
```

```
updateCalculator();

} catch (error) {

  console.error(
    'ERREUR DÉMARRAGE APPLICATION :',
    error
  );

  showAuthPage();

  showMessage(
    "Impossible de démarrer l'application : " +
    getSupabaseErrorMessage(error),
    'error'
  );
}
}
```

```
//  
=====
```

=====

```
// LANCEMENT  
//  
=====
```

=====

```
  
if (  
    document.readyState ===  
    'loading'  
) {  
  
    document.addEventListener(  
        'DOMContentLoaded',  
        boot  
    );  
  
} else {  
  
    boot();  
}
```

```
//  
=====
```

=====

```
// PWA - ENREGISTREMENT DU SERVICE  
WORKER  
//  
=====
```

=====

```
if ('serviceWorker' in navigator) {  
  
  window.addEventListener(  
    'load',  
    () => {  
  
      navigator.serviceWorker  
        .register('sw.js')  
        .catch((error) => {  
  
          console.error(  
            'Erreur service worker :',  
            error
```

```
);  
});  
}  
);  
}
```